INFORME DE AVANCE

# Sistema de comunicación híbrida ESP32-LoRa con interfaz móvil y panel web

Comunicación de texto, alertas SOS y contenido multimedia sin dependencia de internet ni de redes celulares.

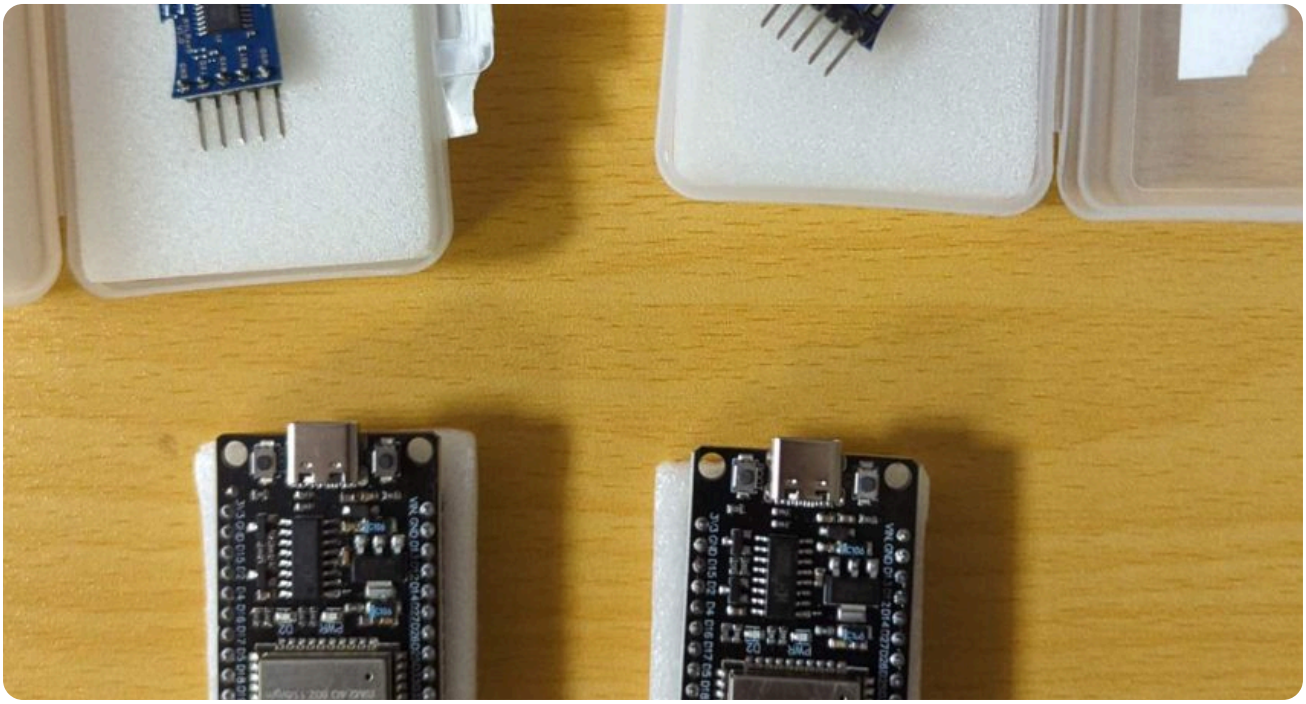


Figura 0. Componentes disponibles al inicio del proyecto.

| | |
|---|---|
| Estudiante(s) | |
| Profesor | |
| Asignatura | |
| Fecha | 28 de octubre de 2026 |
| Página del proyecto | **https://panarepara.com/radio/** |
| Avance estimado | **60 %** (diseño, software y publicación completados; integración en hardware pendiente) |

# Contenido


1 Resumen

2 Objetivos

3 Planteamiento y alcance

4 Materiales

5 Arquitectura del sistema

6 Diseño del hardware

7 Diseño del software

8 Proceso del sistema

9 Resultados a la fecha

10 Estado de avance

11 Limitaciones identificadas

12 Trabajo pendiente

13 Conclusiones parciales

14 Referencias

A Anexo A. Código del firmware

B Anexo B. Código de la interfaz móvil

C Anexo C. Código del panel web


## 1 Resumen


El proyecto desarrolla un sistema de comunicación entre dos estaciones basadas en el microcontrolador ESP32 y el módulo LoRa REYAX RYLR998. Cada estación genera una red WiFi local desde la cual un teléfono accede a una interfaz de chat. El sistema combina dos enlaces de radio: ESP-NOW, de corto alcance y alta velocidad, para texto, fotografías y notas de voz; y LoRa, de largo alcance y baja velocidad, para texto y alertas SOS. La selección entre ambos enlaces es automática y cada mensaje requiere confirmación de entrega.

Una de las estaciones actúa como base: se conecta a internet y retransmite la actividad a un servidor MQTT, lo que permite supervisar el sistema desde un panel web publicado.

Panel web del proyecto (consulta en línea)

**https://panarepara.com/radio/**

Mientras las estaciones no estén encendidas, el panel muestra sus valores en 0 con la indicación “Sin datos por falta de enlace”.

A la fecha se completaron el diseño, el firmware (compilado sin errores), la interfaz móvil, el panel web y su publicación. Quedan pendientes la carga del firmware en las placas y las pruebas de campo.

# 2 Objetivos

## Objetivo general

Desarrollar un sistema de comunicación bidireccional entre dos estaciones que funcione sin infraestructura de internet ni telefonía celular, aprovechando al máximo los componentes disponibles.

## Objetivos específicos

- Integrar el módulo RYLR998 con el ESP32 mediante comandos AT para la transmisión de largo alcance.
- Implementar un enlace ESP-NOW complementario para la transmisión de mayor volumen de datos a corta distancia.
- Diseñar un protocolo con confirmación de entrega, reintentos y cambio automático de enlace.
- Desarrollar una interfaz web servida por el propio ESP32, accesible desde cualquier teléfono sin instalar aplicaciones.
- Incorporar una función de alerta SOS activable desde el teléfono o desde un botón físico.
- Publicar un panel web de supervisión remota mediante MQTT y GitHub Pages.

# 3 Planteamiento y alcance

El proyecto partió de un inventario limitado: dos placas ESP32 y dos módulos de radio. En una primera revisión se identificó que un HC-12 (433 MHz, modulación GFSK) y un RYLR998 (915 MHz, modulación LoRa) no son interoperables, por lo que se confirmó el uso de dos módulos RYLR998.

Se evaluó inicialmente un walkie-talkie con voz en tiempo real y una estación meteorológica. Esta alternativa se postergó porque requiere micrófono, amplificador, parlante y sensor ambiental, componentes no disponibles. Además, la tasa de datos del RYLR998 (del orden de cientos de bytes por segundo) es insuficiente para voz en tiempo real.

El alcance definido fue, por tanto, un sistema híbrido que explota las capacidades existentes: la radio ESP-NOW integrada en el ESP32 para contenido de mayor tamaño y el enlace LoRa para mensajes cortos a gran distancia, utilizando el teléfono como interfaz de entrada y salida.

# 4 Materiales

| Componente | Cant. | Función |
|---|---|---|
| ESP32 DevKit (30 pines, USB-C) | 2 | Procesamiento, red WiFi local, servidor web y enlace ESP-NOW |
| REYAX RYLR998 con antena helicoidal | 2 | Enlace LoRa de largo alcance (915 MHz) |
| Cables dupont hembra-hembra | 8 | Conexión módulo–placa (4 por estación) |
| Batería USB (power bank) | 1 | Alimentación de la estación remota |
| Teléfonos inteligentes | 2 | Interfaz de usuario (equipos existentes) |

| Software y servicios | Uso |
|---|---|
| Arduino IDE / núcleo ESP32 3.3.2 (Espressif) | Desarrollo y compilación del firmware |
| HiveMQ (servidor MQTT público) | Intercambio de mensajes entre la base y el panel web |
| GitHub y GitHub Pages | Control de versiones y publicación del panel |

# 5 Arquitectura del sistema

El sistema se compone de dos estaciones idénticas en hardware. La estación 1 (base) permanece junto al router y la estación 2 opera en campo. Cada una genera su propia red WiFi para el teléfono del usuario.

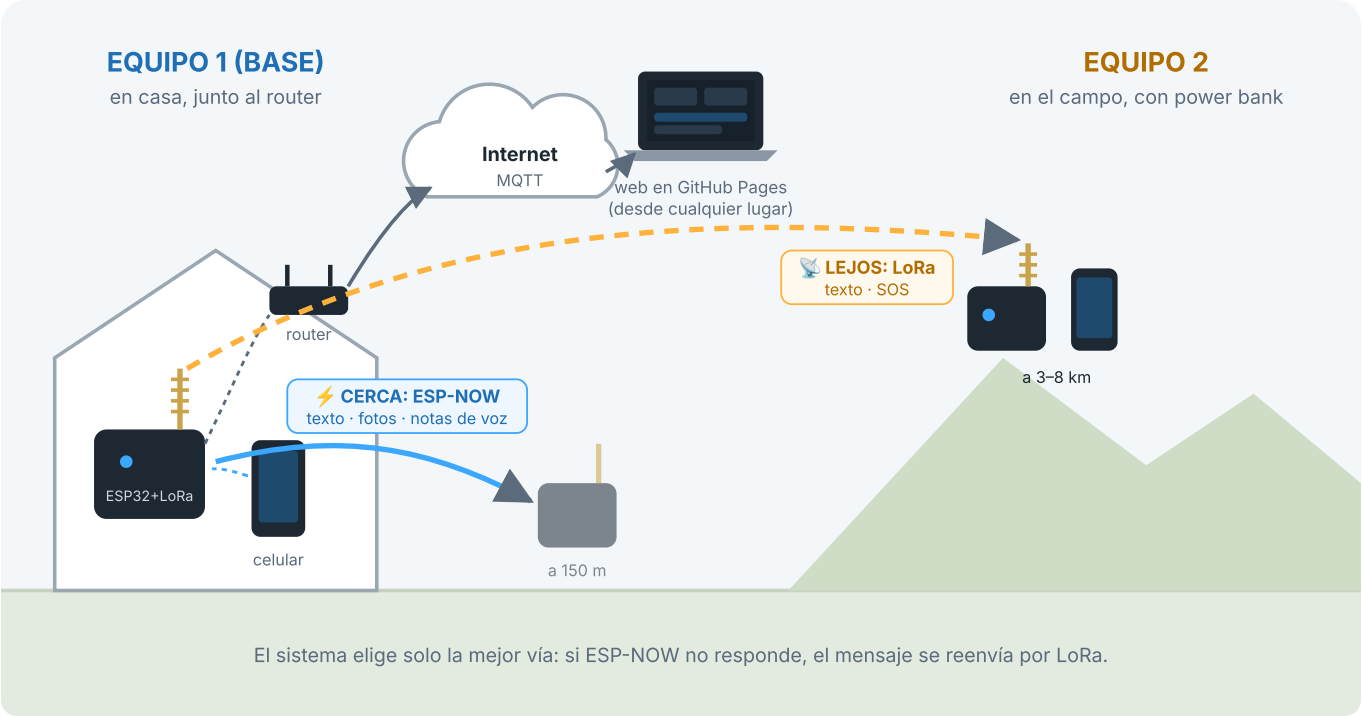



Figura 1. Arquitectura general del sistema.

| Característica | Enlace ESP-NOW | Enlace LoRa |
|---|---|---|
| Tecnología | Radio WiFi del ESP32, sin router | Módulo RYLR998, 915 MHz |
| Alcance esperado | 100–200 m | 1–3 km en zona urbana; 5–10 km con línea de vista |
| Velocidad | Decenas de KB/s | Cientos de bytes por segundo |
| Contenido | Texto, SOS, fotografías y notas de voz | Texto y SOS |
| Seguridad | Sin cifrado | Cifrado por clave compartida (CPIN) |

# 6 Diseño del hardware

La conexión entre el ESP32 y el RYLR998 se resolvió con cuatro conductores sobre el puerto serie UART2. El módulo opera a 3.3 V, por lo que se alimenta desde la salida regulada de la placa. El LED y el botón BOOT incorporados se reutilizan como indicador de alertas y pulsador SOS.




Figura 2. Diagrama de conexiones de cada estación.

| Pin RYLR998 | Pin ESP32 | Descripción |
|---|---|---|
| VDD | 3V3 | Alimentación 3.3 V |
| GND | GND | Referencia común |
| TXD | RX2 (GPIO16) | Datos del módulo hacia el ESP32 |
| RXD | TX2 (GPIO17) | Datos del ESP32 hacia el módulo |
| NRST | — | Sin conexión |

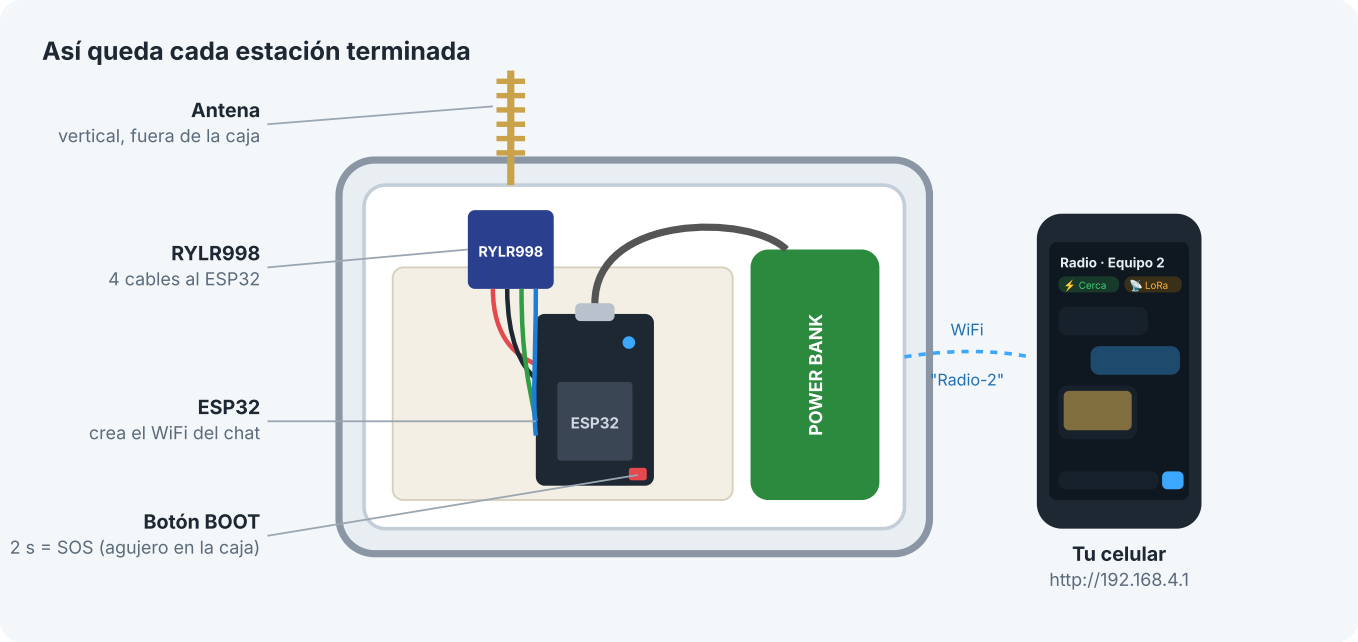



Figura 3. Disposición prevista de cada estación.

# 7 Diseño del software

## 7.1 Organización del firmware

Ambas estaciones ejecutan el mismo programa; la función de cada una se define con dos parámetros de configuración (`EQUIPO` y `ES_BASE`). El firmware se organizó en trece módulos: configuración, control del RYLR998, enlace ESP-NOW, protocolo de mensajes, transferencia de archivos, servidor web, puente MQTT y bucle principal. El código completo y su descripción se incluyen en el Anexo A.

## 7.2 Protocolo y selección de enlace

Cada estación emite un latido por ESP-NOW cada 2 s. Si la otra estación lo recibió en los últimos 8 s, se considera que ambas están cerca y el mensaje se envía por ESP-NOW; en caso contrario, por LoRa. Todo mensaje requiere una confirmación del receptor; si no llega, el sistema reintenta y, de ser necesario, cambia de enlace.

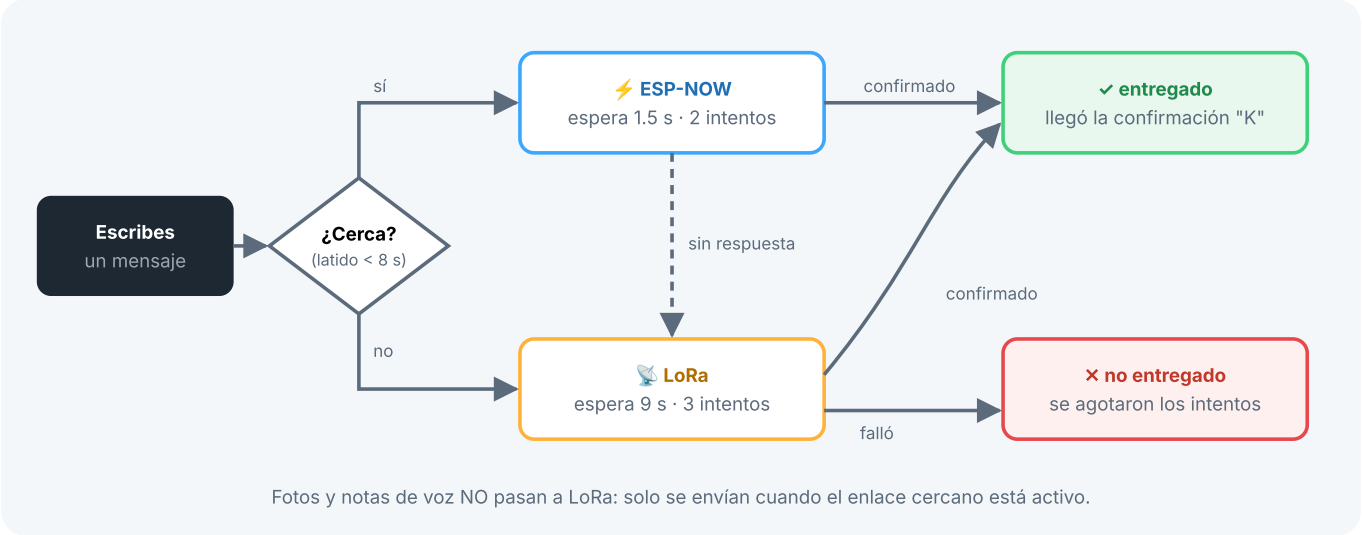



Figura 4. Lógica de envío, confirmación y cambio de enlace.

| Trama | Ejemplo | Significado |
|---|---|---|
| Mensaje | `M,1,857,0,Hola` | Estación 1, mensaje 857, tipo texto |
| Confirmación | `K,2,857` | La estación 2 recibió el mensaje 857 |
| Latido | `P,1` | La estación 1 está activa |
| Archivo | `f` / `d` / `a` | Inicio, fragmento de datos y confirmación (solo ESP-NOW) |

## 7.3 Interfaz móvil

La interfaz es una página web almacenada en el ESP32 y accesible en `http://192.168.4.1` desde la red WiFi de la estación, sin necesidad de instalar aplicaciones. Muestra el estado y la intensidad de señal de cada enlace, el estado de entrega de cada mensaje y permite enviar texto, fotografías, notas de voz y alertas SOS (Anexo B).

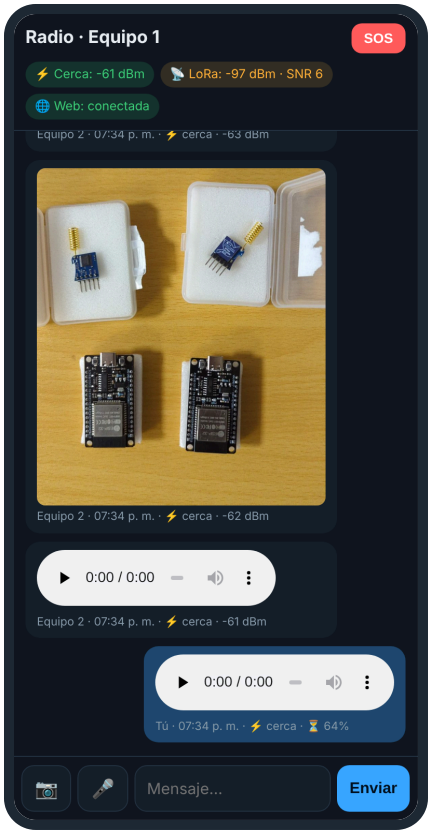



Figura 5. Interfaz en funcionamiento normal (datos simulados).

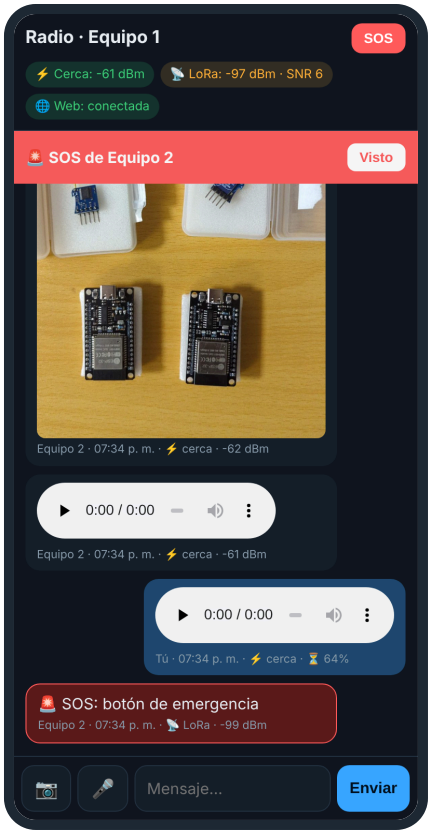



Figura 6. Recepción de una alerta SOS (datos simulados).

## 7.4 Panel web de supervisión

El panel se comunica con el servidor MQTT mediante WebSocket seguro. Presenta el estado de la estación base y de ambos enlaces, el historial de mensajes y permite enviar mensajes y alertas hacia las estaciones (Anexo C).

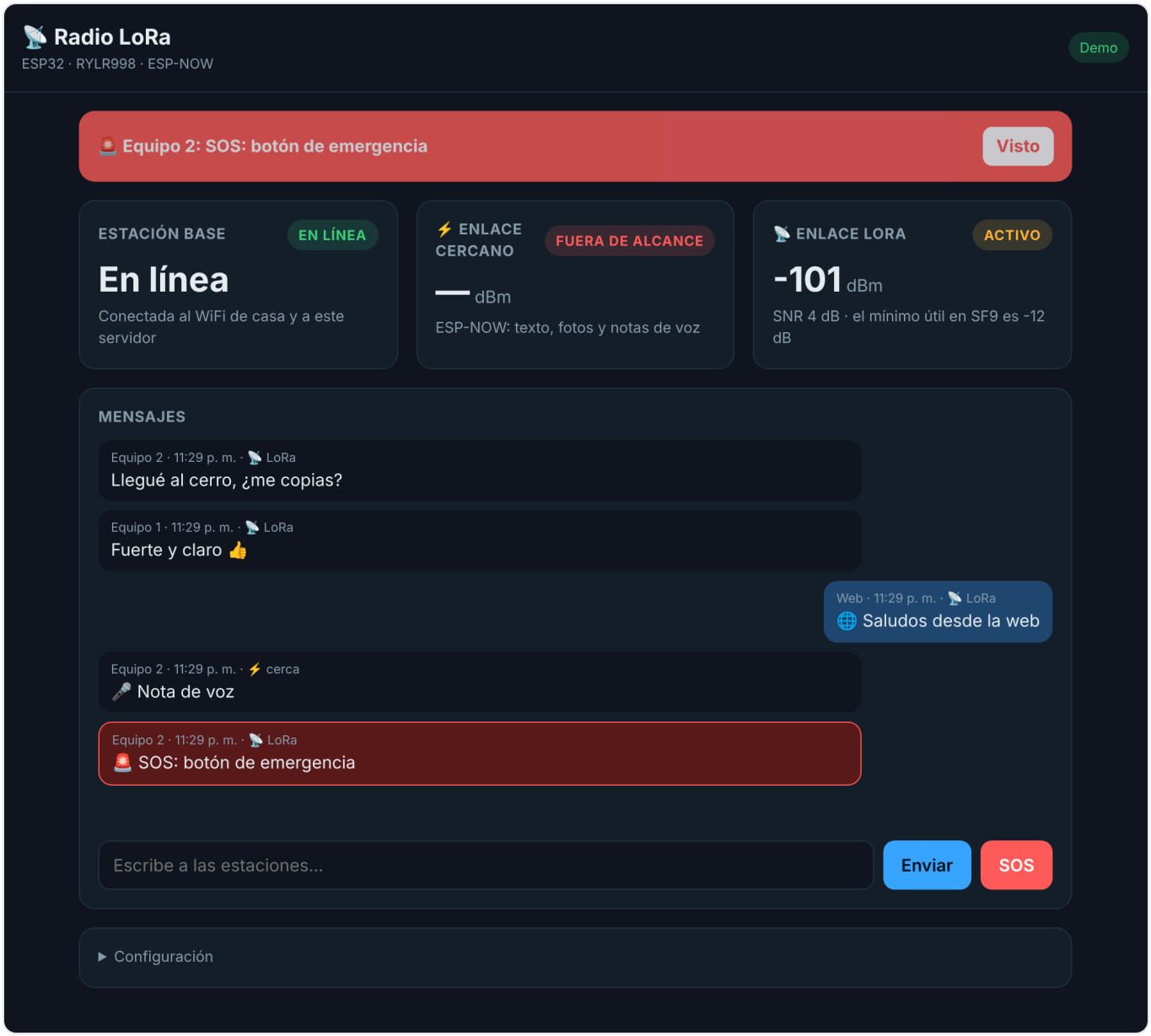



Figura 7. Panel web con datos de demostración (panarepara.com/radio/?demo).

| Tema MQTT | Origen | Contenido |
|---|---|---|
| `prefijo/estado` | Base | Estado de ambos enlaces, cada 10 s |
| `prefijo/radio/msg` | Base | Mensajes transmitidos o recibidos por radio |
| `prefijo/web/msg` · `web/sos` | Panel web | Mensajes y alertas a retransmitir |

# 8 Proceso del sistema

Esta sección registra el desarrollo realizado el 28 de octubre de 2026, desde la evaluación de los componentes hasta la publicación del panel web.

## 8.1 Evaluación inicial

> **18:45 · Identificación del hardware.** Se identificaron dos ESP32 DevKit y dos módulos de radio. Se determinó la incompatibilidad entre HC-12 y RYLR998 y se confirmó la disponibilidad de dos RYLR998.
>
> **19:00 · Primera propuesta.** Walkie-talkie con estación meteorológica. Se postergó por falta de micrófono, amplificador, parlante y sensor.
>
> **19:24 · Propuesta definitiva.** Sistema híbrido ESP-NOW + LoRa con interfaz móvil y panel web, basado únicamente en los componentes disponibles.

## 8.2 Desarrollo y verificación

> **Firmware.** Se compiló con el núcleo ESP32 3.3.2 en sus variantes base y remota, sin errores ni advertencias. Uso de memoria: 77 % del programa y 13 % de la RAM.
>
> **Protocolo LoRa.** Se contrastó con la guía oficial de comandos AT de REYAX (rango de SF, NETWORKID, preámbulo, clave CPIN y carga máxima de 240 bytes) y se reforzó la interpretación de las tramas recibidas.
>
> **Interfaces.** La interfaz móvil y el panel web se validaron en navegador con datos simulados, sin errores de ejecución.

## 8.3 Publicación en GitHub

> **Repositorio.** Se utilizó el repositorio `Companny2/Companny`, que publica el sitio panarepara.com mediante GitHub Pages. El proyecto se ubicó en una carpeta independiente (`/radio`) para no afectar el sitio existente.
>
> **Seguridad.** Se generó un prefijo MQTT aleatorio (`panarepara-radio-acbdcce2`) común al firmware y al panel, y se excluyó la página de los buscadores (`noindex`).
>
> **23:29 · Commit 1e36bff.** Incorporación de la página y del firmware: 3 archivos, 1 105 líneas.

| Archivo | Contenido |
|---|---|
| `radio/index.html` | Panel web (panarepara.com/radio/) |
| `radio/firmware/radio_lora/radio_lora.ino` | Firmware del ESP32 |
| `radio/firmware/radio_lora/pagina.h` | Interfaz móvil |
| `radio/Informe_Avance_Radio_LoRa.pdf` | Este informe |

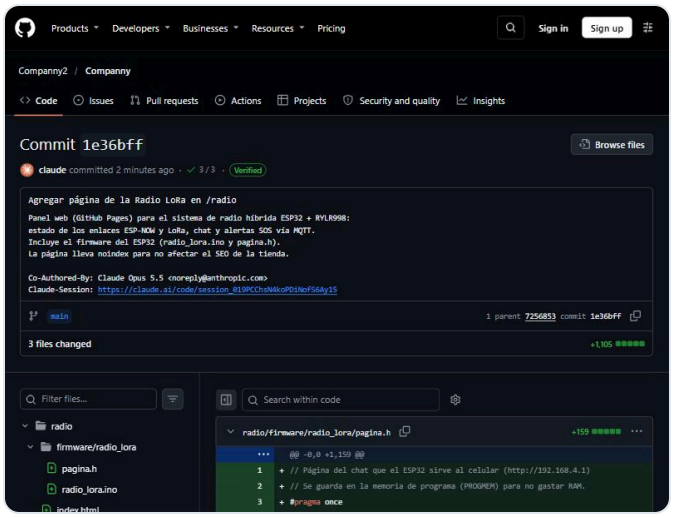



Figura 8. Commit de incorporación en GitHub.

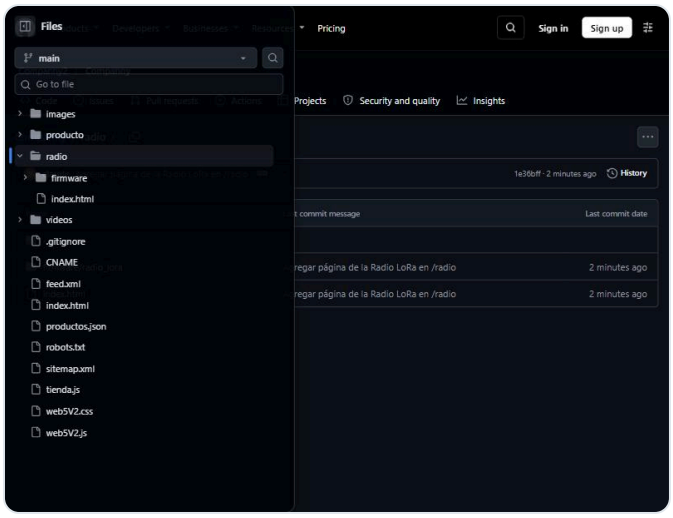



Figura 9. Carpeta `radio` dentro del repositorio.

## 8.4 Verificación de la publicación

La página publicada se verificó en un navegador real. El panel cargó correctamente y estableció conexión con el servidor MQTT; el indicador “Sin datos” es el esperado mientras ninguna estación esté encendida. También se comprobó que el sitio principal no sufrió cambios.

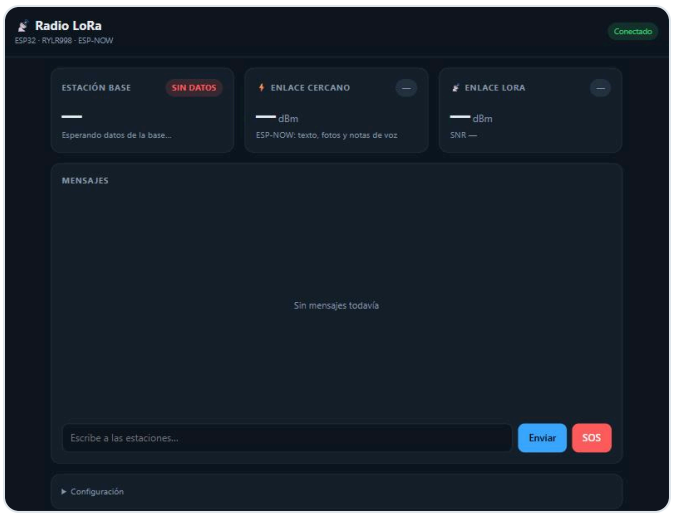



Figura 10. Panel publicado, conectado al servidor MQTT.

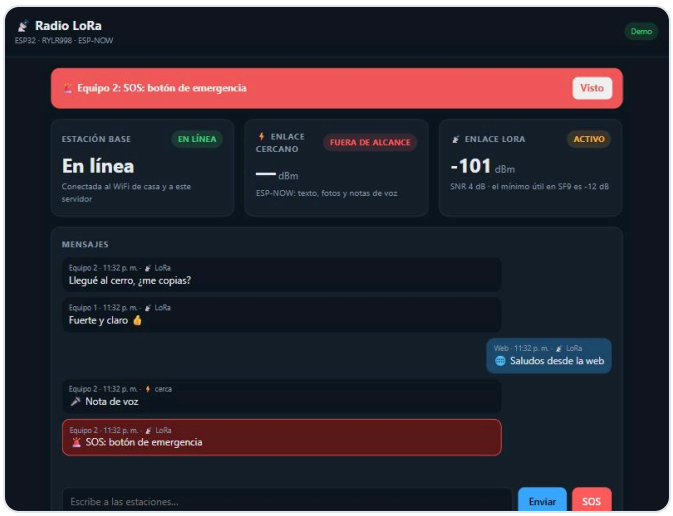



Figura 11. Panel publicado en modo demostración.

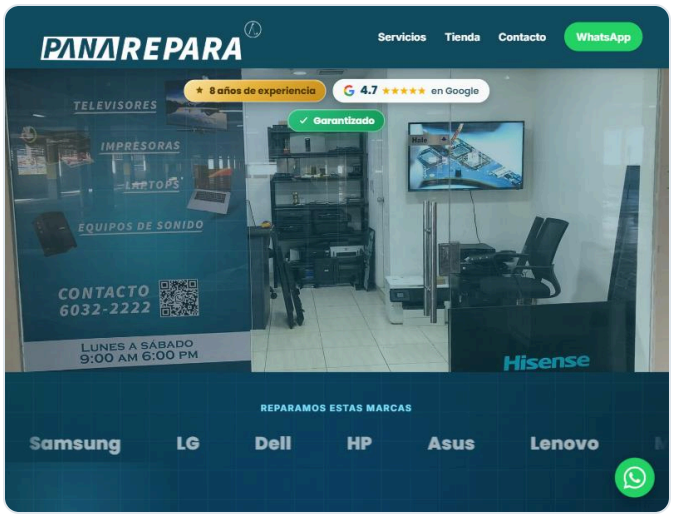



Figura 12. Sitio principal sin alteraciones.

## 8.5 Incidencia detectada y corrección

En la prueba con pantalla móvil (375 px) se observó un desborde horizontal: la cuadrícula principal se ensanchaba hasta el ancho mínimo de la barra de mensajes. Se corrigió con la regla `grid-template-columns: minmax(0,1fr)`, publicada en el commit `6e91ba0` (23:33) y verificada nuevamente en el sitio.

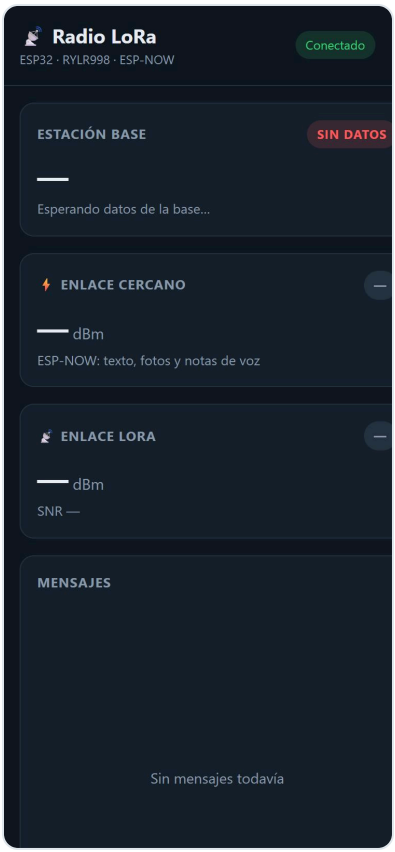



Figura 13. Antes: contenido recortado.




Figura 14. Después de la corrección.

## 8.6 Indicación de ausencia de datos

Para que el panel resulte claro cuando no hay estaciones activas, se modificó su estado inicial: todos los valores se muestran en 0 (0 estaciones, 0 dBm, SNR 0 dB, 0 mensajes), cada enlace se marca como “Sin enlace” y un aviso explica que no hay datos por falta de enlace. Al recibir el primer estado de la base, el aviso desaparece y se muestran los valores reales. El cambio se publicó en el commit `037ec36` (23:40).

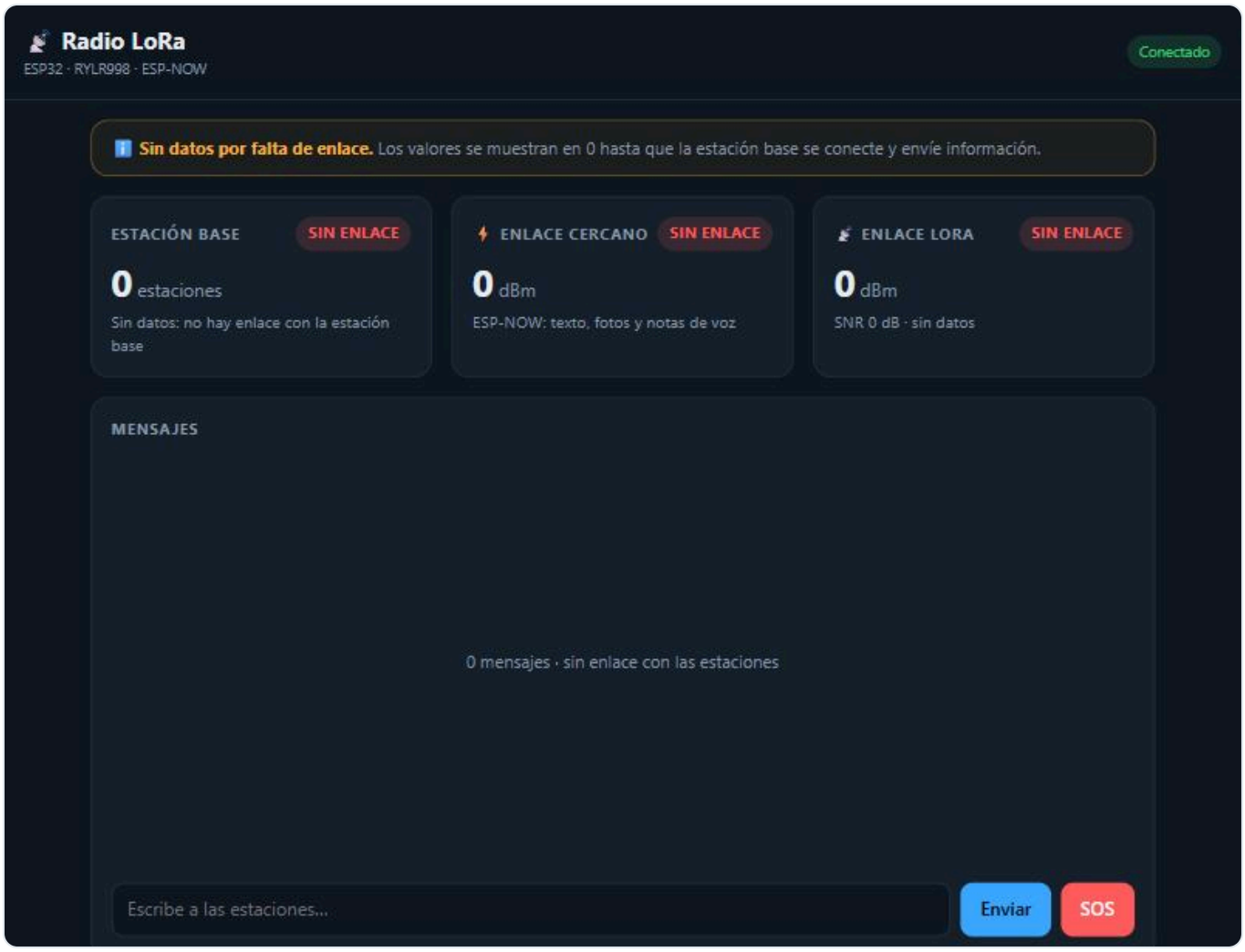


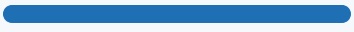
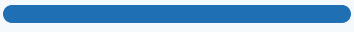
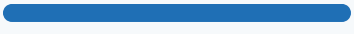

Figura 15. Panel publicado sin enlace: valores en 0 y aviso de ausencia de datos.

Consulta del panel publicado
**https://panarepara.com/radio/**

## 9 Resultados a la fecha

| Indicador | Resultado |
|---|---|
| Compilación del firmware (base y remota) | Sin errores ni advertencias |
| Uso de memoria de programa / RAM | 77 % / 13 % |
| Interfaz móvil con datos simulados | Funcional, sin errores de ejecución |
| Panel web publicado | Disponible en panarepara.com/radio/ y conectado al servidor MQTT |
| Visualización en pantalla móvil | Corregida y verificada (ancho de página = ancho de pantalla) |
| Panel sin estaciones activas | Valores en 0 con aviso “Sin datos por falta de enlace” |
| Impacto sobre el sitio existente | Ninguno |

## 10 Estado de avance

| Etapa | Estado | Avance |
|---|---|---|
| Análisis de componentes y definición del alcance | Completada |  |
| Diseño de arquitectura y hardware | Completada |  |
| Desarrollo del firmware | Completada |  |
| Interfaz móvil y panel web | Completada |  |
| Publicación en GitHub Pages | Completada |  |
| Montaje y carga del firmware en las placas | Pendiente | |
| Pruebas de enlace y de alcance | Pendiente | |
| Ajustes finales e informe definitivo | Pendiente | |
| **Avance global estimado** | **60 %** | |

## 11 Limitaciones identificadas

- La voz en tiempo real no es viable por LoRa debido a su baja tasa de datos; las notas de voz y fotografías dependen del enlace ESP-NOW y, por tanto, de la proximidad entre estaciones.
- El alcance real dependerá del entorno, la altura de las antenas y los obstáculos; los valores indicados son estimaciones a validar en campo.
- El servidor MQTT público no requiere autenticación; la privacidad del panel se basa en un prefijo difícil de adivinar.

- Ambas estaciones deben operar en el mismo canal WiFi; un cambio automático de canal en el router afectaría al enlace ESP-NOW (no al enlace LoRa).
- La frecuencia de 915 MHz y la potencia de 22 dBm deben verificarse frente a la normativa de la autoridad reguladora local.

## 12 Trabajo pendiente

| Actividad | Descripción |
|---|---|
| Montaje | Conexión de los RYLR998 a las placas y verificación de respuesta de ambos módulos. |
| Carga del firmware | Configuración de la estación base (WiFi y canal) y de la estación remota. |
| Prueba de enlace cercano | Transmisión de texto, fotografías y notas de voz por ESP-NOW; medición de tiempos de entrega. |
| Prueba de cambio de enlace | Verificación del paso automático de ESP-NOW a LoRa al aumentar la distancia. |
| Prueba de alcance LoRa | Registro de distancia, RSSI y SNR en distintos puntos. |
| Prueba de SOS y panel | Activación desde el teléfono, el botón físico y el panel web. |

## 13 Conclusiones parciales

- Con los componentes disponibles es posible construir un sistema de comunicación funcional sin internet, combinando dos tecnologías de radio con características complementarias.
- El uso del teléfono como interfaz, mediante una página servida por el ESP32, evita hardware adicional (pantalla, teclado) y el desarrollo de aplicaciones nativas.
- El protocolo con confirmación y cambio automático de enlace brinda robustez ante la pérdida de cobertura del enlace cercano.
- La fase de software y publicación está concluida y verificada; el siguiente hito es la validación en hardware y en campo.

## 14 Referencias


1. REYAX Technology. *RYLR998 / RYLR498 LoRa AT Command Guide.*
2. Espressif Systems. *ESP-NOW — ESP-IDF Programming Guide*. docs.espressif.com
3. Espressif Systems. *Arduino core for the ESP32*, versión 3.3.2. github.com/espressif/arduino-esp32
4. HiveMQ. *Public MQTT Broker*. hivemq.com
5. GitHub. *GitHub Pages documentation*. docs.github.com/pages
6. Panel web del proyecto: https://panarepara.com/radio/

# A Anexo A. Código del firmware

Archivo `radio_lora.ino`, incluido de forma íntegra y organizado en sus trece secciones. La numeración de la izquierda corresponde a las líneas del archivo.

## A.1 Cabecera

Comentario descriptivo del programa: cada estación está formada por un ESP32 y un RYLR998; el teléfono se conecta a la red WiFi de la placa y accede a la interfaz de chat. El enlace cercano usa ESP-NOW y el lejano, LoRa, con selección automática.

```
/*
  ============================================================================
   RADIO HÍBRIDA ESP32 + LoRa (REYAX RYLR998)
  ============================================================================
   Cada estación = 1 ESP32 + 1 RYLR998. Tu celular se conecta al WiFi que
   crea el ESP32 y usa un chat en http://192.168.4.1

   - CERCA  (ESP-NOW, ~100-200 m): texto, notas de voz y fotos.
   - LEJOS  (LoRa, varios km):     texto y SOS.
   - El cambio entre CERCA y LEJOS es automático.
   - Opcional: la estación BASE se conecta al WiFi de tu casa y publica todo
     en una página web (GitHub Pages) mediante MQTT.

   Arduino IDE: placa "ESP32 Dev Module", núcleo ESP32 3.x
   Librería: "PubSubClient" (solo si ES_BASE = 1)
  ============================================================================
*/
```

## A.2 Configuración

Agrupa los parámetros que diferencian a cada estación:

- `EQUIPO` identifica la estación (1 o 2) y es también su dirección LoRa.
- `ES_BASE` habilita en una sola estación la conexión a internet y el puente MQTT.
- `CLAVE_AP` y `CANAL_WIFI` definen la red WiFi local de cada placa. ESP-NOW exige que ambas estaciones operen en el mismo canal.
- Los parámetros LoRa (`LORA_RED`, `LORA_BANDA`, `LORA_PARAM`, `LORA_CLAVE`) deben coincidir en ambas estaciones. Se adoptó la configuración recomendada por el fabricante: SF9, 125 kHz, CR 4/5 y preámbulo 12. `LORA_CLAVE` cifra los mensajes con una contraseña de 8 dígitos hexadecimales.
- Se reutilizan el LED y el botón BOOT incorporados en la placa, sin hardware adicional.

```
// ----------------------------------------------------------------------------
// 1) CONFIGURACIÓN
// ----------------------------------------------------------------------------
#define EQUIPO   1          // 1 en una placa, 2 en la otra
#define ES_BASE  0          // 1 = se conecta al WiFi de casa y a la web (solo UNA placa)

// WiFi que crea la placa para tu celular
const char* CLAVE_AP = "radio1234";        // mínimo 8 caracteres
#define CANAL_WIFI 1                       // igual en las dos placas (ver guía)

// Solo para la BASE
const char* WIFI_SSID    = "TU_WIFI";
const char* WIFI_PASS    = "TU_CLAVE";
const char* MQTT_HOST    = "broker.hivemq.com";
const int   MQTT_PORT    = 1883;
const char* TOPIC_PREFIX = "panarepara-radio-acbdcce2";  // prefijo MQTT único (igual en la web)
```

```
35
36  // LoRa (RYLR998): debe ser igual en las dos placas
37  #define LORA_RX_PIN  16                  // RX2 del ESP32 <- TXD del RYLR998
38  #define LORA_TX_PIN  17                  // TX2 del ESP32 -> RXD del RYLR998
39  #define LORA_BAUD    115200
40  #define LORA_RED     18                  // NETWORKID (0-15 o 18)
41  #define LORA_BANDA   "915000000"         // Hz (verifica la banda permitida en tu país)
42  #define LORA_PARAM   "9,7,1,12"          // SF9, 125 kHz, CR 4/5, preámbulo 12
43  #define LORA_CLAVE   "4F7A21C9"          // cifrado: 8 dígitos hex, igual en ambas
44
45  #define LED_PIN   2                      // LED azul de la placa
46  #define BOTON_SOS 0                       // botón BOOT: pulsación de 2 s = SOS
```

## A.3 Librerías

Se emplean `WebServer` y `DNSServer` para servir la interfaz, `LittleFS` para almacenar fotos y audios en la memoria flash (1.5 MB) y `vector` para listas dinámicas. `PubSubClient` (MQTT) solo se compila en la estación base.

```
48  // ------------------------------------------------------------------------
49  // 2) LIBRERÍAS
50  // ------------------------------------------------------------------------
51  #include <WiFi.h>
52  #include <esp_now.h>
53  #include <esp_wifi.h>
54  #include <WebServer.h>
55  #include <DNSServer.h>
56  #include <LittleFS.h>
57  #include <vector>
58  #include "pagina.h"                      // la página del chat (HTML)
59  #if ES_BASE
60  #include <PubSubClient.h>
61  #endif
```

## A.4 Constantes y tipos

Define los límites del sistema (40 mensajes en memoria, 180 bytes por texto, archivos de hasta 400 KB, fragmentos de 236 bytes) y el umbral de 8 s sin latidos para considerar a la otra estación fuera del alcance de ESP-NOW. La estructura `Msg` representa un mensaje con su autor, tipo (texto, SOS, audio o foto), estado (enviando, entregado, fallido o recibido), vía utilizada y nivel de señal. `Pendiente` registra los textos que esperan confirmación.

```
63  // ------------------------------------------------------------------------
64  // 3) CONSTANTES Y TIPOS
65  // ------------------------------------------------------------------------
66  const uint8_t OTRO = (EQUIPO == 1) ? 2 : 1;   // dirección LoRa de la otra placa
67  const uint8_t BCAST[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
68
69  #define MAX_MSGS        40       // mensajes guardados en memoria
70  #define MAX_TEXTO       180      // bytes por mensaje de texto
71  #define MAX_ARCHIVO     400000   // tamaño máximo de foto / nota de voz
72  #define CHUNK           236      // bytes de archivo por paquete ESP-NOW
73  #define CERCA_MS        8000     // sin noticias ESP-NOW en 8 s = lejos
74  #define LORA_VIVO_MS    75000    // sin noticias LoRa en 75 s = sin enlace
75
76  enum Tipo   : uint8_t { TEXTO = 0, SOS = 1, AUDIO = 2, FOTO = 3 };
77  enum Estado : uint8_t { ENVIANDO = 0, ENTREGADO = 1, FALLIDO = 2, RECIBIDO = 3 };
78
79  struct Msg {
80    uint16_t id;        // número del mensaje (lo pone quien lo envía)
81    uint8_t  de;        // equipo que lo escribió (0 = página web)
82    bool     mio;       // true si salió de esta placa
83    uint8_t  tipo;      // TEXTO, SOS, AUDIO, FOTO
```

```
  uint8_t  estado;    // ENVIANDO, ENTREGADO...
  char     via;       // 'E' ESP-NOW, 'L' LoRa, '-' aún no
  int16_t  rssi;      // señal con la que llegó
  uint8_t  progreso;  // % de envío de archivos
  String   texto;     // texto, o ruta del archivo
};
std::vector<Msg> msgs;
uint32_t version = 1;          // cambia cada vez que algo cambia (para la web)

// Mensajes de texto esperando confirmación
struct Pendiente {
  uint16_t id; uint8_t tipo; String texto;
  char via; uint8_t intentos; uint32_t t;
};
std::vector<Pendiente> pendientes;
```

## A.5 Variables globales

Contiene los objetos del servidor web, del DNS y del puerto serie del módulo LoRa (`Serial2`), las marcas de tiempo de la última comunicación por cada vía (de donde se derivan `cerca()` y `loraVivo()`) y las colas de recepción ESP-NOW y de envío LoRa.

```
// ------------------------------------------------------------------------
// 4) VARIABLES GLOBALES
// ------------------------------------------------------------------------
WebServer server(80);
DNSServer dns;
HardwareSerial& lora = Serial2;

uint16_t siguienteId = 1;
uint32_t ultimoEspNow = 0, ultimoLora = 0;
int espRssi = 0, loraRssi = 0, loraSnr = 0;
bool loraOk = false;
uint8_t sosDe = 0;             // equipo que mandó SOS sin atender (0 = ninguno)
uint32_t sosHasta = 0;

// Cola de recepción ESP-NOW (el callback solo copia, el loop procesa)
struct RxEsp { uint8_t d[250]; int len; int rssi; };
QueueHandle_t colaEsp;

// LoRa: cola de envíos (el módulo atiende un comando a la vez)
std::vector<String> colaLora;
bool loraOcupado = false;
uint32_t loraDesde = 0;
String lineaLora;

#if ES_BASE
WiFiClient net;
PubSubClient mqtt(net);
String T(const char* s) { return String(TOPIC_PREFIX) + "/" + s; }
#endif

bool cerca() { return ultimoEspNow && millis() - ultimoEspNow < CERCA_MS; }
bool loraVivo() { return ultimoLora && millis() - ultimoLora < LORA_VIVO_MS; }
```

## A.6 Lista de mensajes

`buscar()` localiza un mensaje por estación y número; `agregar()` lo incorpora y, al superar el límite, elimina el más antiguo junto con su archivo. `limpiar()` elimina saltos de línea, que el RYLR998 no admite dentro de un mensaje, y recorta textos largos.

```
// ------------------------------------------------------------------------
// 5) LISTA DE MENSAJES
// ------------------------------------------------------------------------
```

```
Msg* buscar(uint8_t de, uint16_t id, bool mio) {
  for (auto& m : msgs)
    if (m.id == id && m.de == de && m.mio == mio) return &m;
  return nullptr;
}

Msg& agregar(const Msg& m) {
  if (msgs.size() >= MAX_MSGS) {                // lleno: borrar el más viejo
    Msg& v = msgs.front();
    if ((v.tipo == AUDIO || v.tipo == FOTO) && v.texto.length())
      LittleFS.remove(v.texto);
    msgs.erase(msgs.begin());
  }
  msgs.push_back(m);
  version++;
  return msgs.back();
}

void parpadear(int veces) {
  for (int i = 0; i < veces; i++) {
    digitalWrite(LED_PIN, HIGH); delay(40);
    digitalWrite(LED_PIN, LOW);  delay(40);
  }
}

String limpiar(String s) {                       // sin saltos de línea
  s.replace("\r", " "); s.replace("\n", " ");
  while (s.length() > MAX_TEXTO) s.remove(s.length() - 1);
  return s;
}
```

## A.7 Control del módulo LoRa

El RYLR998 se controla mediante comandos AT por puerto serie. `loraIniciar()` verifica la respuesta del módulo y configura dirección, red, frecuencia, parámetros de radio, clave de cifrado y potencia (22 dBm); la clave se reenvía en cada arranque porque el módulo la borra al reiniciarse. `loraEnviar()` genera comandos del tipo `AT+SEND=2,13,M,1,857,0,Hola`. `loraLoop()` envía un comando a la vez y procesa las recepciones con formato `+RCV=dirección,largo,datos,RSSI,SNR`.

```
// ---------------------------------------------------------------------------
// 6) LoRa (RYLR998 con comandos AT)
// ---------------------------------------------------------------------------
// Envía un comando y espera "+OK" (solo se usa al arrancar)
bool loraComando(const String& cmd, uint32_t espera = 1500) {
  while (lora.available()) lora.read();
  lora.print(cmd + "\r\n");
  uint32_t t0 = millis();
  String l;
  while (millis() - t0 < espera) {
    while (lora.available()) {
      char c = lora.read();
      if (c == '\n') {
        l.trim();
        if (l.startsWith("+OK")) return true;
        if (l.startsWith("+ERR")) { Serial.printf("LoRa %s -> %s\n", cmd.c_str(), l.c_str()); return fals
e; }
        l = "";
      } else l += c;
    }
  }
  return false;
}

void loraIniciar() {
```

```
  lora.begin(LORA_BAUD, SERIAL_8N1, LORA_RX_PIN, LORA_TX_PIN);
  delay(200);
  loraOk = loraComando("AT");
  if (!loraOk) {
    Serial.println("!! El RYLR998 no responde: revisa TXD->16, RXD->17, 3V3 y GND");
    return;
  }
  loraComando("AT+ADDRESS=" + String(EQUIPO));
  loraComando("AT+NETWORKID=" + String(LORA_RED));
  loraComando("AT+BAND=" + String(LORA_BANDA));
  loraComando("AT+PARAMETER=" + String(LORA_PARAM));
  loraComando("AT+CPIN=" + String(LORA_CLAVE));
  loraComando("AT+CRFOP=22");                    // potencia máxima (22 dBm)
  Serial.println("LoRa listo");
}

// Encola un envío LoRa a la otra placa
void loraEnviar(const String& datos) {
  if (!loraOk) return;
  colaLora.push_back("AT+SEND=" + String(OTRO) + "," + String(datos.length()) + "," + datos);
}

void procesarTramaTexto(const String& s, char via, int rssi);

// Atiende el módulo: manda lo pendiente y lee lo que llega
void loraLoop() {
  if (!loraOk) return;
  if (!loraOcupado && !colaLora.empty()) {
    lora.print(colaLora.front() + "\r\n");
    colaLora.erase(colaLora.begin());
    loraOcupado = true;
    loraDesde = millis();
  }
  if (loraOcupado && millis() - loraDesde > 5000) loraOcupado = false;  // por si acaso

  while (lora.available()) {
    char c = lora.read();
    if (c != '\n') { if (lineaLora.length() < 300) lineaLora += c; continue; }
    lineaLora.trim();
    if (lineaLora.startsWith("+OK") || lineaLora.startsWith("+ERR")) {
      loraOcupado = false;
    } else if (lineaLora.startsWith("+RCV=")) {
      // Formato: +RCV=<dirección>,<largo>,<datos>,<RSSI>,<SNR>
      int p1 = lineaLora.indexOf(',');
      int p2 = lineaLora.indexOf(',', p1 + 1);
      int largo = lineaLora.substring(p1 + 1, p2).toInt();
      int ini = p2 + 1;
      while (ini < (int)lineaLora.length() && lineaLora[ini] == ' ') ini++;  // por si trae espacios
      String datos = lineaLora.substring(ini, ini + largo);
      String resto = lineaLora.substring(ini + largo + 1);
      int p3 = resto.indexOf(',');
      loraRssi = resto.substring(0, p3).toInt();
      loraSnr  = resto.substring(p3 + 1).toInt();
      ultimoLora = millis();
      version++;
      procesarTramaTexto(datos, 'L', loraRssi);
    }
    lineaLora = "";
  }
}
```

## A.8 Enlace ESP-NOW

La función de recepción se ejecuta en la tarea interna del WiFi, por lo que únicamente copia el paquete a una cola; el procesamiento ocurre en el bucle principal. Los envíos se dirigen a la dirección de difusión, lo que

evita configurar la dirección MAC de la otra placa.

```
// ---------------------------------------------------------------------------
// 7) ESP-NOW (radio directa entre ESP32)
// ---------------------------------------------------------------------------
void onEspNow(const esp_now_recv_info_t* info, const uint8_t* data, int len) {
  if (len <= 0 || len > 250) return;
  RxEsp r;
  memcpy(r.d, data, len);
  r.len = len;
  r.rssi = info->rx_ctrl ? info->rx_ctrl->rssi : 0;
  xQueueSend(colaEsp, &r, 0);                    // el loop lo procesa
}

void espEnviar(const uint8_t* d, size_t n) { esp_now_send(BCAST, d, n); }
void espEnviarTexto(const String& s) { espEnviar((const uint8_t*)s.c_str(), s.length()); }
```

## A.9 Protocolo de texto

Se diseñó un formato común para ambas vías, legible y compacto:

- `M,1,857,0,Hola` : mensaje de la estación 1, número 857, tipo 0 (texto; 1 = SOS).
- `K,2,857` : confirmación de recepción del mensaje 857.
- `P,1` : latido de presencia.

El receptor confirma siempre por la misma vía por la que recibió y descarta duplicados. `revisarPendientes()` implementa la lógica de reintentos y cambio de vía descrita en la sección 7.2.

```
// ---------------------------------------------------------------------------
// 8) PROTOCOLO DE TEXTO (igual por ESP-NOW y por LoRa)
//    M,<de>,<id>,<tipo>,<texto>   mensaje
//    K,<de>,<id>                  confirmación ("lo recibí")
//    P,<de>                       "estoy aquí" (latido)
// ---------------------------------------------------------------------------
void enviarPorVia(char via, const String& trama) {
  if (via == 'E') espEnviarTexto(trama);
  else loraEnviar(trama);
}

void publicarMsg(const Msg& m);   // (MQTT, más abajo)

void procesarTramaTexto(const String& s, char via, int rssi) {
  if (s.length() < 3 || s[1] != ',') return;
  char tipoTrama = s[0];
  int c1 = s.indexOf(',', 2);
  uint8_t de = s.substring(2, c1 < 0 ? s.length() : c1).toInt();
  if (de == EQUIPO) return;                      // eco de mí mismo

  if (tipoTrama == 'P') return;                  // latido: ya se anotó la señal

  int c2 = s.indexOf(',', c1 + 1);
  uint16_t id = s.substring(c1 + 1, c2 < 0 ? s.length() : c2).toInt();

  if (tipoTrama == 'K') {                        // confirmación de algo mío
    for (size_t i = 0; i < pendientes.size(); i++)
      if (pendientes[i].id == id) { pendientes.erase(pendientes.begin() + i); break; }
    Msg* m = buscar(EQUIPO, id, true);
    if (!m) m = buscar(0, id, true);
    if (m && m->estado != ENTREGADO) { m->estado = ENTREGADO; m->via = via; version++; }
    return;
  }

  if (tipoTrama == 'M') {
    int c3 = s.indexOf(',', c2 + 1);
    if (c3 < 0) return;
```

```
    uint8_t tipo = s.substring(c2 + 1, c3).toInt();
    String texto = s.substring(c3 + 1);
    // Confirmar SIEMPRE por la misma vía (aunque sea repetido)
    enviarPorVia(via, "K," + String(EQUIPO) + "," + String(id));
    if (buscar(de, id, false)) return;            // repetido: ya lo tengo
    Msg m{id, de, false, tipo, RECIBIDO, via, (int16_t)rssi, 100, texto};
    agregar(m);
    publicarMsg(m);
    Serial.printf("[%s %s] %s\n", via == 'E' ? "CERCA" : "LoRa",
                  tipo == SOS ? "SOS" : "MSG", texto.c_str());
    if (tipo == SOS) { sosDe = de; sosHasta = millis() + 120000; }
    parpadear(3);
  }
}

// Envía un mensaje de texto o SOS, eligiendo la mejor vía
void enviarMensaje(uint8_t tipo, String texto, uint8_t de = EQUIPO) {
  texto = limpiar(texto);
  if (!texto.length()) return;
  uint16_t id = siguienteId++;
  char via = cerca() ? 'E' : 'L';
  Msg m{id, de, true, tipo, ENVIANDO, via, 0, 0, texto};
  agregar(m);
  publicarMsg(m);
  String trama = "M," + String(EQUIPO) + "," + String(id) + "," + String(tipo) + "," + texto;
  enviarPorVia(via, trama);
  pendientes.push_back({id, tipo, trama, via, 1, millis()});
  Serial.printf("[ENVIADO por %s] %s\n", via == 'E' ? "CERCA" : "LoRa", texto.c_str());
}

// Reintentos: por ESP-NOW espera 1.5 s (2 intentos); si no hay respuesta,
// pasa a LoRa y espera 9 s (3 intentos). Si nada funciona: "no entregado".
void revisarPendientes() {
  for (size_t i = 0; i < pendientes.size(); i++) {
    Pendiente& p = pendientes[i];
    uint32_t espera = (p.via == 'E') ? 1500 : 9000;
    if (millis() - p.t < espera) continue;

    bool reintentar = true;
    if (p.via == 'E') {
      if (p.intentos < 2 && cerca()) p.intentos++;            // otra vez cerca
      else if (loraOk) { p.via = 'L'; p.intentos = 1; }       // saltar a LoRa
      else reintentar = false;
    } else {
      if (p.intentos < 3) p.intentos++;
      else reintentar = false;
    }

    Msg* m = buscar(EQUIPO, p.id, true);
    if (!m) m = buscar(0, p.id, true);
    if (!reintentar) {
      if (m) { m->estado = FALLIDO; version++; }
      pendientes.erase(pendientes.begin() + i);
      i--;
      continue;
    }
    enviarPorVia(p.via, p.texto);
    p.t = millis();
    if (m && m->via != p.via) { m->via = p.via; version++; }
  }
}
```

## A.10 Transferencia de archivos

Las fotos y notas de voz se transmiten por ESP-NOW en fragmentos con confirmación individual (método de parada y espera): una trama de inicio (**f**) con tamaño y extensión, tramas de datos (**d**) de 236 bytes y

confirmaciones (**a**) que indican el siguiente fragmento esperado. Un fragmento sin confirmación se repite a los 80 ms y la transferencia se declara fallida tras 60 fallos consecutivos. El receptor escribe directamente en la memoria flash, sin consumir RAM.

```
// ---------------------------------------------------------------------------
// 9) ARCHIVOS (fotos y notas de voz) — solo por ESP-NOW, en trozos
//    f <id:2> <tipo:1> <tamaño:4> <ext>   inicio
//    d <id:2> <n:2> <datos>               trozo número n
//    a <id:2> <siguiente:2>               "ya tengo hasta n-1, mándame n"
// ---------------------------------------------------------------------------
struct Envio {
  bool activo = false; uint16_t id; uint8_t tipo; File f; uint32_t tam;
  uint16_t total, n; uint32_t t; uint16_t fallos; String ext;
} tx;
struct Recibo {
  bool activo = false; uint16_t id; uint8_t de; uint8_t tipo; File f;
  uint32_t tam, escritos; uint16_t siguiente; String ruta; uint32_t t; int rssi;
} rx;

void txMandarTrozo() {
  uint8_t b[250];
  if (tx.n == 0xFFFF) {                          // todavía no aceptó el inicio
    b[0] = 'f'; memcpy(b + 1, &tx.id, 2); b[3] = tx.tipo; memcpy(b + 4, &tx.tam, 4);
    size_t e = tx.ext.length();
    memcpy(b + 8, tx.ext.c_str(), e);
    espEnviar(b, 8 + e);
  } else {
    b[0] = 'd'; memcpy(b + 1, &tx.id, 2); memcpy(b + 3, &tx.n, 2);
    tx.f.seek((uint32_t)tx.n * CHUNK);
    size_t r = tx.f.read(b + 5, CHUNK);
    espEnviar(b, 5 + r);
  }
  tx.t = millis();
}

bool empezarEnvioArchivo(const String& ruta, uint8_t tipo, const String& ext) {
  if (tx.activo || !cerca()) return false;
  tx.f = LittleFS.open(ruta, "r");
  if (!tx.f) return false;
  tx.activo = true; tx.id = siguienteId++; tx.tipo = tipo; tx.tam = tx.f.size();
  tx.total = (tx.tam + CHUNK - 1) / CHUNK; tx.n = 0xFFFF; tx.fallos = 0; tx.ext = ext;
  Msg m{tx.id, EQUIPO, true, tipo, ENVIANDO, 'E', 0, 0, ruta};
  agregar(m);
  publicarMsg(m);
  txMandarTrozo();
  return true;
}

void txTerminar(bool ok) {
  tx.f.close();
  tx.activo = false;
  Msg* m = buscar(EQUIPO, tx.id, true);
  if (m) { m->estado = ok ? ENTREGADO : FALLIDO; m->progreso = ok ? 100 : m->progreso; version++; }
}

void txLoop() {
  if (!tx.activo) return;
  if (millis() - tx.t > 80) {                    // sin respuesta: repetir
    if (++tx.fallos > 60) { txTerminar(false); return; }
    txMandarTrozo();
  }
}

void procesarArchivo(const uint8_t* d, int len, int rssi) {
  if (len < 3) return;
  uint16_t id; memcpy(&id, d + 1, 2);
```

```
  if (d[0] == 'a' && tx.activo && id == tx.id && len >= 5) {   // soy el que envía
    uint16_t sig; memcpy(&sig, d + 3, 2);
    tx.fallos = 0;
    if (sig >= tx.total) { txTerminar(true); return; }
    if (tx.n == 0xFFFF || sig > tx.n) {
      tx.n = sig;
      Msg* m = buscar(EQUIPO, tx.id, true);
      uint8_t p = (uint32_t)sig * 100 / tx.total;
      if (m && m->progreso != p) { m->progreso = p; version++; }
    }
    txMandarTrozo();
    return;
  }
  uint8_t r[5] = {'a'};
  if (d[0] == 'f' && len >= 8) {                    // soy el que recibe: inicio
    if (!(rx.activo && rx.id == id)) {
      if (rx.activo) { rx.f.close(); LittleFS.remove(rx.ruta); }
      uint32_t tam; memcpy(&tam, d + 4, 4);
      String ext((const char*)(d + 8), len - 8);
      rx.activo = true; rx.id = id; rx.tipo = d[3]; rx.tam = tam; rx.escritos = 0;
      rx.siguiente = 0; rx.rssi = rssi;
      rx.ruta = "/m/r" + String(id) + "_" + String(millis() % 100000) + "." + ext;
      rx.f = LittleFS.open(rx.ruta, "w");
    }
    rx.t = millis();
    memcpy(r + 1, &id, 2); memcpy(r + 3, &rx.siguiente, 2);
    espEnviar(r, 5);
    return;
  }
  // (si ya terminó y el emisor repite el último trozo, solo se vuelve a confirmar)
  if (d[0] == 'd' && len >= 5 && id == rx.id && (rx.activo || rx.siguiente > 0)) {
    uint16_t n; memcpy(&n, d + 3, 2);
    if (rx.activo && n == rx.siguiente) {
      rx.f.write(d + 5, len - 5);
      rx.escritos += len - 5;
      rx.siguiente++;
    }
    rx.t = millis();
    memcpy(r + 1, &id, 2); memcpy(r + 3, &rx.siguiente, 2);
    espEnviar(r, 5);
    if (rx.activo && rx.escritos >= rx.tam) {      // ¡completo!
      rx.f.close();
      rx.activo = false;
      if (!buscar(OTRO, id, false)) {
        Msg m{id, OTRO, false, rx.tipo, RECIBIDO, 'E', (int16_t)rx.rssi, 100, rx.ruta};
        agregar(m);
        publicarMsg(m);
        parpadear(3);
        Serial.printf("[CERCA] %s recibida (%u bytes)\n", rx.tipo == FOTO ? "Foto" : "Nota de voz", rx.ta
m);
      }
    }
  }
}

void rxLoop() {                                   // recepción abandonada
  if (rx.activo && millis() - rx.t > 8000) {
    rx.f.close(); LittleFS.remove(rx.ruta); rx.activo = false;
  }
}
```

## A.11 Servidor web de la interfaz

El ESP32 publica la interfaz (`/`), el estado en formato JSON (`/api`), la recepción de mensajes y SOS (`/enviar`), la carga de archivos (`/subir`, almacenados en flash mientras llegan) y la descarga de archivos

recibidos (`/f/...`). También responde las rutas de comprobación de conectividad de Android e iOS para evitar que el teléfono abandone la red por carecer de internet.

```
// ---------------------------------------------------------------------------
// 10) PÁGINA WEB DEL CELULAR
// ---------------------------------------------------------------------------
String jsonEsc(const String& s) {
  String o; o.reserve(s.length() + 8);
  for (size_t i = 0; i < s.length(); i++) {
    char c = s[i];
    if (c == '"' || c == '\\') { o += '\\'; o += c; }
    else if ((uint8_t)c < 0x20) o += ' ';
    else o += c;
  }
  return o;
}

void apiEstado() {
  String j; j.reserve(4096);
  j = "{\"v\":" + String(version) + ",\"yo\":" + String(EQUIPO) + ",\"otro\":" + String(OTRO);
  j += ",\"cerca\":{\"on\":" + String(cerca() ? "true" : "false") + ",\"rssi\":" + String(espRssi) + "}";
  j += ",\"lora\":{\"ok\":" + String(loraOk ? "true" : "false") + ",\"on\":" + String(loraVivo() ? "true"
: "false") +
       ",\"rssi\":" + String(loraRssi) + ",\"snr\":" + String(loraSnr) + "}";
  j += ",\"ocupado\":" + String(tx.activo ? "true" : "false");
  j += ",\"sos\":" + String(sosDe);
#if ES_BASE
  j += ",\"web\":" + String(mqtt.connected() ? "true" : "false");
#endif
  j += ",\"msgs\":[";
  for (size_t i = 0; i < msgs.size(); i++) {
    const Msg& m = msgs[i];
    if (i) j += ",";
    j += "{\"id\":" + String(m.id) + ",\"de\":" + String(m.de) + ",\"mio\":" + (m.mio ? "true" : "false")
+
         ",\"t\":" + String(m.tipo) + ",\"e\":" + String(m.estado) + ",\"via\":\"" + String(m.via) +
         "\",\"rssi\":" + String(m.rssi) + ",\"p\":" + String(m.progreso) +
         ",\"x\":\"" + jsonEsc((m.tipo == AUDIO || m.tipo == FOTO) ? String("/f") + m.texto : m.texto) +
"\"}";
  }
  j += "]}";
  server.sendHeader("Cache-Control", "no-store");
  server.send(200, "application/json", j);
}

void apiEnviar() {
  String t = server.arg("t");
  uint8_t tipo = server.arg("sos") == "1" ? SOS : TEXTO;
  if (tipo == SOS && !t.length()) t = "SOS: necesito ayuda";
  enviarMensaje(tipo, t);
  server.send(200, "text/plain", "ok");
}

void apiSosVisto() { sosDe = 0; version++; server.send(200, "text/plain", "ok"); }

// Subida de foto / audio desde el celular
File subida; String subidaRuta; bool subidaMal = false;
void apiSubidaDatos() {
  HTTPUpload& u = server.upload();
  if (u.status == UPLOAD_FILE_START) {
    String ext = server.arg("ext");
    if (ext.length() < 2 || ext.length() > 5) ext = "bin";
    subidaRuta = "/m/t" + String(millis() % 100000) + "." + ext;
    subida = LittleFS.open(subidaRuta, "w");
    subidaMal = !subida;
  } else if (u.status == UPLOAD_FILE_WRITE) {
```

```
    if (!subidaMal) {
      if (subida.size() + u.currentSize > MAX_ARCHIVO) subidaMal = true;
      else subida.write(u.buf, u.currentSize);
    }
  } else if (u.status == UPLOAD_FILE_END || u.status == UPLOAD_FILE_ABORTED) {
    if (subida) subida.close();
    if (u.status == UPLOAD_FILE_ABORTED) subidaMal = true;
  }
}
void apiSubidaFin() {
  uint8_t tipo = server.arg("tipo") == "foto" ? FOTO : AUDIO;
  String ext = server.arg("ext");
  if (subidaMal) { LittleFS.remove(subidaRuta); server.send(413, "text/plain", "Archivo muy grande (máx. 4
00 KB)"); return; }
  if (!empezarEnvioArchivo(subidaRuta, tipo, ext)) {
    LittleFS.remove(subidaRuta);
    server.send(409, "text/plain", "La otra estación está lejos u ocupada: fotos y audios solo viajan CERC
A");
    return;
  }
  server.send(200, "text/plain", "ok");
}

void apiArchivo() {
  String ruta = server.uri().substring(2);         // quita "/f"
  if (!ruta.startsWith("/m/") || !LittleFS.exists(ruta)) { server.send(404, "text/plain", "no"); return; }
  String ct = "application/octet-stream";
  if (ruta.endsWith(".jpg")) ct = "image/jpeg";
  else if (ruta.endsWith(".m4a") || ruta.endsWith(".mp4")) ct = "audio/mp4";
  else if (ruta.endsWith(".aac")) ct = "audio/aac";
  else if (ruta.endsWith(".amr")) ct = "audio/amr";
  else if (ruta.endsWith(".3gp")) ct = "audio/3gpp";
  else if (ruta.endsWith(".ogg") || ruta.endsWith(".opus")) ct = "audio/ogg";
  else if (ruta.endsWith(".webm")) ct = "audio/webm";
  else if (ruta.endsWith(".mp3")) ct = "audio/mpeg";
  else if (ruta.endsWith(".wav")) ct = "audio/wav";
  File f = LittleFS.open(ruta, "r");
  server.streamFile(f, ct);
  f.close();
}

void iniciarWeb() {
  server.on("/", []() { server.send_P(200, "text/html", PAGINA); });
  server.on("/api", apiEstado);
  server.on("/enviar", HTTP_POST, apiEnviar);
  server.on("/sosvisto", HTTP_POST, apiSosVisto);
  server.on("/subir", HTTP_POST, apiSubidaFin, apiSubidaDatos);
  // Respuestas para que el celular no crea que "no hay internet" y se desconecte
  server.on("/generate_204", []() { server.send(204); });
  server.on("/hotspot-detect.html", []() { server.send(200, "text/html", "Success"); });
  server.onNotFound([]() {
    if (server.uri().startsWith("/f/")) { apiArchivo(); return; }
    server.sendHeader("Location", "http://192.168.4.1/");
    server.send(302);
  });
  server.begin();
  dns.start(53, "*", WiFi.softAPIP());             // cualquier dirección -> esta placa
}
```

## A.12 Puente MQTT

En la estación base, los mensajes escritos en el panel web se reciben por MQTT y se retransmiten por radio identificados como “Web”. La base publica cada 10 s el estado de ambos enlaces y registra un mensaje de última voluntad para indicar su desconexión. Las fotos y audios se publican solo como descripción, ya que no se transmiten por internet.

```
// ---------------------------------------------------------------------------
// 11) MQTT (solo BASE): puente con la página de GitHub
// ---------------------------------------------------------------------------
#if ES_BASE
void onMqtt(char* topic, byte* payload, unsigned int len) {
  String s; for (unsigned i = 0; i < len; i++) s += (char)payload[i];
  bool sos = String(topic).endsWith("/web/sos");
  if (!s.length()) s = "SOS";
  enviarMensaje(sos ? SOS : TEXTO, "🌐 " + s, 0);   // 🌐 = escrito en la web
}

void mqttLoop() {
  if (WiFi.status() != WL_CONNECTED) return;
  if (!mqtt.connected()) {
    static uint32_t intento = 0;
    if (millis() - intento < 5000) return;
    intento = millis();
    String id = "radio-" + String((uint32_t)ESP.getEfuseMac(), HEX);
    if (!mqtt.connect(id.c_str(), T("estado").c_str(), 0, true, "{\"base\":false}")) return;
    mqtt.subscribe(T("web/msg").c_str());
    mqtt.subscribe(T("web/sos").c_str());
    Serial.println("MQTT conectado");
  }
  mqtt.loop();
  static uint32_t ultimo = 0;
  if (millis() - ultimo > 10000) {
    ultimo = millis();
    String j = "{\"base\":true,\"cerca\":{\"on\":" + String(cerca() ? "true" : "false") + ",\"rssi\":" + S
tring(espRssi) +
               "},\"lora\":{\"on\":" + String(loraVivo() ? "true" : "false") + ",\"rssi\":" + String(loraR
ssi) +
               ",\"snr\":" + String(loraSnr) + "}}";
    mqtt.publish(T("estado").c_str(), j.c_str(), true);
  }
}
#endif

void publicarMsg(const Msg& m) {
#if ES_BASE
  if (!mqtt.connected()) return;
  String x = (m.tipo == AUDIO) ? "🎤 Nota de voz" : (m.tipo == FOTO) ? "📷 Foto" : m.texto;
  String j = "{\"de\":" + String(m.de) + ",\"id\":" + String(m.id) + ",\"t\":" + String(m.tipo) +
             ",\"via\":\"" + String(m.via) + "\",\"x\":\"" + jsonEsc(x) + "\"}";
  mqtt.publish(T("radio/msg").c_str(), j.c_str());
#endif
}
```

## A.13 Inicialización (setup)

Al encender, la estación limpia la memoria de archivos, configura el módulo LoRa, establece la conexión WiFi (en la base, primero con el router para adoptar su canal, dado que el ESP32 posee una sola radio WiFi), crea su red local, inicia ESP-NOW y el servidor web.

```
// ---------------------------------------------------------------------------
// 12) SETUP
// ---------------------------------------------------------------------------
void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.printf("\n=== RADIO EQUIPO %d %s===\n", EQUIPO, ES_BASE ? "(BASE) " : "");
  pinMode(LED_PIN, OUTPUT);
  pinMode(BOTON_SOS, INPUT_PULLUP);
  siguienteId = esp_random() % 50000 + 1;         // al azar: evita repetidos tras reiniciar

  // Memoria para fotos y audios (se borra al encender)
```

```
  if (!LittleFS.begin(true)) Serial.println("!! Error de memoria LittleFS");
  File dir = LittleFS.open("/m");
  if (dir) {
    std::vector<String> borrar;
    for (File f = dir.openNextFile(); f; f = dir.openNextFile()) borrar.push_back(String("/m/") + f.name
());
    for (auto& r : borrar) LittleFS.remove(r);
  } else LittleFS.mkdir("/m");

  loraIniciar();

  // WiFi: punto de acceso para el celular (+ conexión a casa si es BASE)
  WiFi.mode(WIFI_AP_STA);
  uint8_t canal = CANAL_WIFI;
#if ES_BASE
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Conectando al WiFi de casa");
  for (int i = 0; i < 60 && WiFi.status() != WL_CONNECTED; i++) { delay(250); Serial.print("."); }
  Serial.println();
  if (WiFi.status() == WL_CONNECTED) {
    canal = WiFi.channel();
    Serial.printf("WiFi OK. >>> CANAL %d: ponlo en CANAL_WIFI de la otra placa\n", canal);
  } else Serial.println("Sin WiFi de casa: la radio funciona igual, sin la web");
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(onMqtt);
  mqtt.setBufferSize(512);
#endif
  String nombre = "Radio-" + String(EQUIPO);
  WiFi.softAP(nombre.c_str(), CLAVE_AP, canal);
  esp_wifi_set_ps(WIFI_PS_NONE);
  Serial.printf("WiFi para el celular: %s / %s -> abre http://192.168.4.1\n", nombre.c_str(), CLAVE_AP);

  // ESP-NOW
  colaEsp = xQueueCreate(24, sizeof(RxEsp));
  if (esp_now_init() == ESP_OK) {
    esp_now_register_recv_cb(onEspNow);
    esp_now_peer_info_t peer = {};
    memcpy(peer.peer_addr, BCAST, 6);
    peer.channel = 0;
    peer.ifidx = WIFI_IF_AP;
    peer.encrypt = false;
    esp_now_add_peer(&peer);
  } else Serial.println("!! Error iniciando ESP-NOW");

  iniciarWeb();
  Serial.println("Listo.");
}
```

## A.14 Bucle principal (loop)

En cada ciclo se atienden la interfaz web, los paquetes ESP-NOW recibidos, el módulo LoRa, los reintentos y las transferencias. Se envían latidos cada 2 s por ESP-NOW y cada ~30 s por LoRa (con un desfase según la estación para reducir colisiones), se supervisa el botón SOS y se controla el LED de alerta. Los textos ingresados por el monitor serie también se transmiten, lo que facilita las pruebas sin teléfono.

```
// ---------------------------------------------------------------------------
// 13) LOOP
// ---------------------------------------------------------------------------
void loop() {
  dns.processNextRequest();
  server.handleClient();

  // Lo que llegó por ESP-NOW
  RxEsp r;
  while (xQueueReceive(colaEsp, &r, 0) == pdTRUE) {
```

```
    bool antes = cerca();
    ultimoEspNow = millis();
    espRssi = r.rssi;
    if (!antes) version++;
    char c = r.d[0];
    if (c == 'f' || c == 'd' || c == 'a') procesarArchivo(r.d, r.len, r.rssi);
    else procesarTramaTexto(String((const char*)r.d, r.len), 'E', r.rssi);
  }

  loraLoop();
  revisarPendientes();
  txLoop();
  rxLoop();

  // Latidos: "estoy aquí"
  static uint32_t latidoEsp = 0, latidoLora = 0;
  if (millis() - latidoEsp > 2000) { latidoEsp = millis(); espEnviarTexto("P," + String(EQUIPO)); }
  if (millis() - latidoLora > 30000 + EQUIPO * 1500) { latidoLora = millis(); loraEnviar("P," + String(EQU
IPO)); }

  // ¿Se perdió el enlace cercano? avisar a la web
  static bool cercaAntes = false;
  if (cerca() != cercaAntes) { cercaAntes = cerca(); version++; }

  // Botón BOOT: pulsación de 2 s = SOS
  static uint32_t pulsado = 0;
  if (digitalRead(BOTON_SOS) == LOW) {
    if (!pulsado) pulsado = millis();
    else if (pulsado != 1 && millis() - pulsado > 2000) {
      enviarMensaje(SOS, "SOS: botón de emergencia");
      pulsado = 1;                                  // no repetir hasta soltar
    }
  } else pulsado = 0;

  // LED: parpadeo rápido si hay un SOS sin atender
  if (sosDe && millis() < sosHasta) digitalWrite(LED_PIN, (millis() / 120) % 2);
  else if (sosDe) { sosDe = 0; digitalWrite(LED_PIN, LOW); }

  // Mensajes escritos en el Monitor Serie
  static String linea;
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') { if (linea.length()) enviarMensaje(TEXTO, linea); linea = ""; }
    else linea += c;
  }

#if ES_BASE
  mqttLoop();
#endif
}
```

# B Anexo B. Código de la interfaz móvil

Archivo `pagina.h`, incluido de forma íntegra.

## B.1 Declaración

La interfaz completa se almacena como una cadena en la memoria de programa (`PROGMEM`) mediante un literal de texto sin escapes.

```
// Página del chat que el ESP32 sirve al celular (http://192.168.4.1)
// Se guarda en la memoria de programa (PROGMEM) para no gastar RAM.
#pragma once
const char PAGINA[] PROGMEM = R"PAGINA(
<!doctype html><html lang="es"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1">
<title>Radio</title>
```

## B.2 Estilos

Tema oscuro tipo aplicación de mensajería: lista de mensajes desplazable, barra de escritura fija, mensajes propios en azul y alertas SOS en rojo.

```
<style>
:root{--bg:#0f1720;--card:#17212c;--line:#253241;--tx:#e8eef4;--mu:#8a9bab;--yo:#1e4a6e;--ok:#3fd17a;--ba
d:#ff5d5d;--lo:#ffb23b}
*{box-sizing:border-box}html,body{height:100%}
body{margin:0;background:var(--bg);color:var(--tx);font:15px/1.4 system-ui,-apple-system,Roboto,sans-seri
f;display:flex;flex-direction:column}
header{padding:10px 12px;border-bottom:1px solid var(--line);display:flex;flex-direction:column;gap:8px}
.top{display:flex;align-items:center;justify-content:space-between}
h1{font-size:17px;margin:0}
.chips{display:flex;gap:6px;flex-wrap:wrap}
.chip{font-size:12px;padding:4px 9px;border-radius:99px;background:var(--line);color:var(--mu)}
.chip.on{background:rgba(63,209,122,.15);color:var(--ok)}
.chip.lo{background:rgba(255,178,59,.15);color:var(--lo)}
#sosBtn{background:var(--bad);color:#fff;border:0;border-radius:10px;padding:7px 12px;font-weight:700}
#alerta{display:none;background:var(--bad);color:#fff;padding:12px;font-weight:700;justify-content:space-b
etween;align-items:center;animation:p 1s infinite}
#alerta button{background:#fff;color:var(--bad);border:0;border-radius:8px;padding:6px 12px;font-weight:70
0}
@keyframes p{50%{opacity:.75}}
#lista{flex:1;overflow-y:auto;padding:12px;display:flex;flex-direction:column;gap:8px}
.m{max-width:82%;padding:8px 11px;border-radius:14px;background:var(--card);word-wrap:break-word}
.m.yo{align-self:flex-end;background:var(--yo)}
.m.sos{background:#5c1a1a;border:1px solid var(--bad)}
.m small{display:block;font-size:11px;color:var(--mu);margin-top:3px}
.m img{max-width:100%;border-radius:8px;display:block}
.m audio{width:230px;max-width:100%}
.vacio{color:var(--mu);text-align:center;margin:auto;font-size:14px}
form{display:flex;gap:6px;padding:8px;border-top:1px solid var(--line)}
input[type=text]{flex:1;min-width:0;background:var(--card);border:1px solid var(--line);color:var(--tx);bo
rder-radius:12px;padding:11px;font:inherit}
.b{background:var(--card);border:1px solid var(--line);color:var(--tx);border-radius:12px;padding:0 12px;f
ont-size:18px}
.b:disabled{opacity:.35}
#env{background:#3ba7ff;color:#04121f;border:0;font-weight:700;font-size:15px}
#aviso{display:none;position:fixed;left:12px;right:12px;bottom:70px;background:#000c;padding:10px 12px;bor
der-radius:10px;font-size:14px}
</style></head><body>
```

## B.3 Estructura

Encabezado con el número de estación, botón SOS e indicadores de estado de cada enlace; franja de alarma; lista de mensajes; barra con botones de foto, nota de voz, campo de texto y envío. Dos campos de archivo ocultos acceden a la cámara y a la grabadora del teléfono.

```
<header>
  <div class="top"><h1 id="tit">Radio</h1><button id="sosBtn">SOS</button></div>
  <div class="chips"><span id="cCerca" class="chip">⚡ Cerca: —</span><span id="cLora" class="chip">📡 LoR
a: —</span><span id="cWeb" class="chip" style="display:none">🌐 Web</span></div>
</header>
<div id="alerta"><span id="alertaTxt">🚨 SOS</span><button id="visto">Visto</button></div>
<div id="lista"><div class="vacio" id="vacio">Aún no hay mensajes.<br>Escribe abajo para empezar.</div></d
iv>
<div id="aviso"></div>
<form id="f">
  <button type="button" class="b" id="bFoto" title="Foto">📷</button>
  <button type="button" class="b" id="bAudio" title="Nota de voz">🎤</button>
  <input type="text" id="txt" maxlength="160" placeholder="Mensaje…" autocomplete="off">
  <button class="b" id="env">Enviar</button>
</form>
<input type="file" id="iFoto" accept="image/*" capture="environment" hidden>
<input type="file" id="iAudio" accept="audio/*" capture hidden>
<script>
```

## B.4 Utilidades

Funciones auxiliares de búsqueda de elementos, avisos flotantes y registro de la hora de llegada (el ESP32 no dispone de reloj de tiempo real).

```
// --- utilidades ---
const $ = id => document.getElementById(id);
let estado = null, vistos = {}, nodos = {};

function aviso(t, ms = 3500) { const a = $("aviso"); a.textContent = t; a.style.display = "block"; clearTi
meout(a._t); a._t = setTimeout(() => a.style.display = "none", ms); }
function nombre(n) { return n == 0 ? "Web" : "Equipo " + n; }
function hora(k) { if (!vistos[k]) vistos[k] = new Date(); return vistos[k].toLocaleTimeString("es-PA", {
hour: "2-digit", minute: "2-digit" }); }
```

## B.5 Representación de mensajes

Cada mensaje se presenta como burbuja de texto, imagen o reproductor de audio, con una línea de autor, hora, vía y estado. El contenido se inserta como texto plano para impedir la inyección de código.

```
// --- cómo se dibuja cada mensaje ---
function pie(m) {
  const via = m.via == "E" ? "⚡ cerca" : m.via == "L" ? "📡 LoRa" : "";
  let est = "";
  if (m.mio) est = m.e == 0 ? (m.t >= 2 ? "⏳ " + m.p + "%" : "⏳ enviando") : m.e == 1 ? "✓ entregado" :
"× no entregado";
  else est = m.rssi ? m.rssi + " dBm" : "";
  return [m.mio ? "Tú" : nombre(m.de), hora(m.mio + "-" + m.de + "-" + m.id), via, est].filter(Boolean).jo
in(" · ");
}

function crear(m) {
  const d = document.createElement("div");
  d.className = "m" + (m.mio ? " yo" : "") + (m.t == 1 ? " sos" : "");
  if (m.t == 3) { const i = document.createElement("img"); i.src = m.x; i.onload = () => { const l = $("li
sta"); l.scrollTop = l.scrollHeight; }; d.append(i); }
  else if (m.t == 2) { const a = document.createElement("audio"); a.controls = true; a.preload = "none";
a.src = m.x; d.append(a); }
```

```
  else { const s = document.createElement("div"); s.textContent = (m.t == 1 ? "🚨 " : "") + m.x; d.append
(s); }
  const p = document.createElement("small"); d.append(p); d._pie = p;
  return d;
}
```

## B.6 Actualización del estado

Se actualizan los indicadores, se habilitan los botones multimedia solo con enlace cercano, se muestra la alarma SOS con vibración y se agregan los mensajes nuevos sin redibujar los existentes, de modo que un audio en reproducción no se interrumpe.

```
// --- dibujar el estado completo ---
function pintar(j) {
  $("tit").textContent = "Radio · Equipo " + j.yo;
  const c = $("cCerca"); c.textContent = "⚡ Cerca: " + (j.cerca.on ? j.cerca.rssi + " dBm" : "fuera de alc
ance"); c.className = "chip" + (j.cerca.on ? " on" : "");
  const l = $("cLora");
  l.textContent = "📡 LoRa: " + (!j.lora.ok ? "sin módulo" : j.lora.on ? j.lora.rssi + " dBm · SNR " + j.l
ora.snr : "esperando…");
  l.className = "chip" + (j.lora.on ? " lo" : "");
  if ("web" in j) { const w = $("cWeb"); w.style.display = ""; w.className = "chip" + (j.web ? " on" :
""); w.textContent = "🌐 Web: " + (j.web ? "conectada" : "sin conexión"); }
  const medios = j.cerca.on && !j.ocupado;
  $("bFoto").disabled = $("bAudio").disabled = !medios;

  const al = $("alerta");
  if (j.sos) { $("alertaTxt").textContent = "🚨 SOS de " + nombre(j.sos); al.style.display = "flex"; if (n
avigator.vibrate) navigator.vibrate([300, 150, 300]); }
  else al.style.display = "none";

  const lista = $("lista"), vivos = {};
  let abajo = lista.scrollHeight - lista.scrollTop - lista.clientHeight < 60;
  for (const m of j.msgs) {
    const k = m.mio + "-" + m.de + "-" + m.id; vivos[k] = 1;
    if (!nodos[k]) { nodos[k] = crear(m); lista.append(nodos[k]); abajo = true; }
    nodos[k]._pie.textContent = pie(m);
  }
  for (const k in nodos) if (!vivos[k]) { nodos[k].remove(); delete nodos[k]; }
  $("vacio").style.display = j.msgs.length ? "none" : "";
  if (abajo) lista.scrollTop = lista.scrollHeight;
}
```

## B.7 Consulta periódica

La interfaz solicita el estado a la placa cada segundo y señala la pérdida de conexión cuando no hay respuesta.

```
// --- preguntar a la placa cada segundo ---
async function actualizar() {
  try { const r = await fetch("/api"); estado = await r.json(); pintar(estado); }
  catch (e) { $("cCerca").textContent = "⚠ Sin conexión con la placa"; }
  setTimeout(actualizar, 1000);
}
```

## B.8 Envío de texto y SOS

Los mensajes se envían por formulario; el SOS requiere confirmación previa.

```
// --- enviar texto y SOS ---
function post(url, datos) { return fetch(url, { method: "POST", body: new URLSearchParams(datos) }); }

$("f").onsubmit = e => {
  e.preventDefault();
```

```
  const t = $("txt").value.trim(); if (!t) return;
  post("/enviar", { t }); $("txt").value = "";
};
$("sosBtn").onclick = () => { if (confirm("¿Enviar SOS a la otra estación?")) post("/enviar", { sos: 1, t:
"" }); };
$("visto").onclick = () => post("/sosvisto", {});
```

## B.9 Fotos y notas de voz

Las fotos se reducen a 800 px y se comprimen en JPEG en el propio navegador (40–80 KB). Las notas de voz se cargan con el formato de la grabadora del teléfono, con un máximo de 400 KB. La carga muestra su porcentaje de avance.

```
// --- fotos y notas de voz ---
function subir(blob, tipo, ext) {
  if (blob.size > 400000) { aviso("Muy grande (" + Math.round(blob.size / 1024) + " KB). Máximo 400 KB: gr
aba menos tiempo."); return; }
  const fd = new FormData(); fd.append("archivo", blob, "a." + ext);
  const x = new XMLHttpRequest();
  x.open("POST", "/subir?tipo=" + tipo + "&ext=" + ext);
  x.upload.onprogress = e => aviso("Cargando… " + Math.round(e.loaded * 100 / e.total) + "%", 2000);
  x.onload = () => aviso(x.status == 200 ? "Enviando por radio…" : x.responseText);
  x.onerror = () => aviso("Error al cargar el archivo");
  x.send(fd);
}

$("bFoto").onclick = () => $("iFoto").click();
$("bAudio").onclick = () => $("iAudio").click();
$("iFoto").onchange = e => {
  const f = e.target.files[0]; e.target.value = ""; if (!f) return;
  const img = new Image();
  img.onload = () => {   // reducir la foto a 800 px y comprimir en JPEG
    const k = Math.min(1, 800 / Math.max(img.width, img.height));
    const c = document.createElement("canvas"); c.width = img.width * k; c.height = img.height * k;
    c.getContext("2d").drawImage(img, 0, 0, c.width, c.height);
    c.toBlob(b => subir(b, "foto", "jpg"), "image/jpeg", 0.6);
    URL.revokeObjectURL(img.src);
  };
  img.src = URL.createObjectURL(f);
};
$("iAudio").onchange = e => {
  const f = e.target.files[0]; e.target.value = ""; if (!f) return;
  let ext = (f.name.split(".").pop() || "m4a").toLowerCase().slice(0, 5);
  subir(f, "audio", ext);
};
actualizar();
</script></body></html>
)PAGINA";
```

# C Anexo C. Código del panel web

Archivo `radio/index.html` del repositorio, incluido de forma íntegra.

## C.1 Encabezado

Documento HTML del panel, con indicación `noindex` para excluirlo de los buscadores y carga de la librería MQTT.js.

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="robots" content="noindex, nofollow">
<title>Radio LoRa</title>
<script src="https://unpkg.com/mqtt@5/dist/mqtt.min.js"></script>
```

## C.2 Estilos

Mismo lenguaje visual que la interfaz del teléfono; tarjetas adaptables a pantallas de escritorio y móviles.

```
<style>
  :root{--bg:#0f1720;--card:#17212c;--line:#253241;--tx:#e8eef4;--mu:#8a9bab;--yo:#1e4a6e;--ok:#3fd17a;--b
ad:#ff5d5d;--lo:#ffb23b;--az:#3ba7ff}
  *{box-sizing:border-box}
  body{margin:0;background:var(--bg);color:var(--tx);font:15px/1.45 system-ui,-apple-system,"Segoe UI",Rob
oto,sans-serif}
  header{display:flex;align-items:center;justify-content:space-between;gap:12px;padding:16px;border-botto
m:1px solid var(--line)}
  h1{font-size:19px;margin:0}
  h1 small{display:block;font-size:12px;color:var(--mu);font-weight:400}
  .pill{font-size:12px;padding:5px 10px;border-radius:99px;background:var(--line);color:var(--mu);white-sp
ace:nowrap}
  .pill.on{background:rgba(63,209,122,.15);color:var(--ok)}
  .pill.lo{background:rgba(255,178,59,.15);color:var(--lo)}
  .pill.off{background:rgba(255,93,93,.15);color:var(--bad)}
  main{max-width:900px;margin:0 auto;padding:16px;display:grid;grid-template-columns:minmax(0,1fr);gap:16p
x}
  .grid{min-width:0}
  .grid{display:grid;gap:16px;grid-template-columns:repeat(auto-fit,minmax(240px,1fr))}
  .card{background:var(--card);border:1px solid var(--line);border-radius:14px;padding:16px}
  .card h2{margin:0 0 10px;font-size:13px;color:var(--mu);text-transform:uppercase;letter-spacing:.6px;dis
play:flex;justify-content:space-between;align-items:center}
  .big{font-size:30px;font-weight:700;line-height:1.1}
  .big span{font-size:15px;color:var(--mu);font-weight:400}
  .sub{font-size:13px;color:var(--mu);margin-top:6px}
  #aviso{padding:12px 16px;font-size:14px;color:var(--mu);border-color:rgba(255,178,59,.4);background:rgba
(255,178,59,.06)}
  #aviso b{color:var(--lo)}
  #alerta{display:none;background:var(--bad);color:#fff;border-radius:14px;padding:14px 16px;font-weight:7
00;align-items:center;justify-content:space-between;animation:p 1s infinite}
  #alerta button{background:#fff;color:var(--bad);border:0;border-radius:8px;padding:6px 12px;font-weight:
700}
  @keyframes p{50%{opacity:.75}}
  #log{height:340px;overflow-y:auto;display:flex;flex-direction:column;gap:8px;margin-bottom:10px}
  .m{max-width:80%;padding:8px 11px;border-radius:12px;background:var(--bg)}
  .m small{display:block;font-size:11px;color:var(--mu)}
  .m.web{align-self:flex-end;background:var(--yo)}
  .m.sos{background:#5c1a1a;border:1px solid var(--bad)}
  .vacio{color:var(--mu);margin:auto;font-size:14px}
  form{display:flex;gap:8px}
  input{flex:1;min-width:0;width:0;background:var(--bg);border:1px solid var(--line);color:var(--tx);borde
```

```
r-radius:10px;padding:10px 12px;font:inherit}
  button{border:0;border-radius:10px;padding:10px 16px;font:inherit;font-weight:600;cursor:pointer}
  #env{background:var(--az);color:#04121f}
  #sos{background:var(--bad);color:#fff}
  details{font-size:13px;color:var(--mu)}
  details input{margin-top:6px;width:100%}
</style>
```

## C.3 Estructura

Franja de alarma, tarjetas de estado (base, enlace cercano y enlace LoRa), chat con envío y SOS, y panel de configuración del prefijo.

```
</head>
<body>
<header>
  <h1>📡 Radio LoRa<small>ESP32 · RYLR998 · ESP-NOW</small></h1>
  <span id="conn" class="pill">Conectando…</span>
</header>

<main>
  <div id="aviso" class="card">ℹ️ <b>Sin datos por falta de enlace.</b> Los valores se muestran en 0 hasta
que la estación base se conecte y envíe información.</div>
  <div id="alerta"><span id="alertaTxt">🚨 SOS</span><button id="visto">Visto</button></div>

  <div class="grid">
    <section class="card">
      <h2>Estación base <span id="stBase" class="pill off">Sin enlace</span></h2>
      <div class="big" id="baseTxt">0<span> estaciones</span></div>
      <div class="sub" id="baseSub">Sin datos: no hay enlace con la estación base</div>
    </section>
    <section class="card">
      <h2>⚡ Enlace cercano <span id="stCerca" class="pill off">Sin enlace</span></h2>
      <div class="big" id="cercaRssi">0<span> dBm</span></div>
      <div class="sub">ESP-NOW: texto, fotos y notas de voz</div>
    </section>
    <section class="card">
      <h2>📡 Enlace LoRa <span id="stLora" class="pill off">Sin enlace</span></h2>
      <div class="big" id="loraRssi">0<span> dBm</span></div>
      <div class="sub" id="loraSnr">SNR 0 dB · sin datos</div>
    </section>
  </div>

  <section class="card">
    <h2>Mensajes</h2>
    <div id="log"><div class="vacio" id="vacio">0 mensajes · sin enlace con las estaciones</div></div>
    <form id="f">
      <input id="txt" maxlength="150" placeholder="Escribe a las estaciones…" autocomplete="off">
      <button id="env">Enviar</button>
      <button type="button" id="sos">SOS</button>
    </form>
  </section>

  <details class="card">
    <summary>Configuración</summary>
    <label>Prefijo (igual al TOPIC_PREFIX del ESP32)<input id="prefix"></label>
  </details>
</main>

<script>
```

## C.4 Configuración

Servidor MQTT mediante WebSocket seguro, requerido por el protocolo HTTPS de GitHub Pages, y prefijo de temas. El parámetro `?demo` carga datos de demostración.

```
const BROKER = "wss://broker.hivemq.com:8884/mqtt";
const DEFAULT_PREFIX = "panarepara-radio-acbdcce2";
const DEMO = new URLSearchParams(location.search).has("demo");
const $ = id => document.getElementById(id);
```

## C.5 Prefijo

El prefijo puede modificarse y se conserva en el navegador.

```
// --- prefijo guardado en el navegador ---
let prefix = DEFAULT_PREFIX;
try { prefix = localStorage.getItem("prefix") || DEFAULT_PREFIX; } catch (e) {}
$("prefix").value = prefix;
$("prefix").addEventListener("change", () => {
  try { localStorage.setItem("prefix", $("prefix").value.trim()); } catch (e) {}
  location.reload();
});
```

## C.6 Estado de los enlaces

Presenta el estado publicado por la base cada 10 s; tras 30 s sin datos, la base se indica como “Sin datos”.

```
// --- estado de los enlaces ---
let ultimoEstado = 0;
function pill(el, on, txtOn, txtOff, cls = "on") { el.textContent = on ? txtOn : txtOff; el.className = "p
ill " + (on ? cls : "off"); }
// Sin enlace: todos los valores en 0
function sinEnlace() {
  $("aviso").style.display = "";
  pill($("stBase"), false, "", "Sin enlace");
  $("baseTxt").innerHTML = "0<span> estaciones</span>";
  $("baseSub").textContent = "Sin datos: no hay enlace con la estación base";
  pill($("stCerca"), false, "", "Sin enlace");
  $("cercaRssi").innerHTML = "0<span> dBm</span>";
  pill($("stLora"), false, "", "Sin enlace");
  $("loraRssi").innerHTML = "0<span> dBm</span>";
  $("loraSnr").textContent = "SNR 0 dB · sin datos";
}
function mostrarEstado(e, vivo) {
  if (!vivo) return;                        // un dato guardado no indica enlace actual
  ultimoEstado = Date.now();
  if (!e.base) { sinEnlace(); return; }
  $("aviso").style.display = "none";
  pill($("stBase"), true, "En línea", "");
  $("baseTxt").innerHTML = "En línea";
  $("baseSub").textContent = "Conectada al WiFi de casa y a este servidor";
  pill($("stCerca"), e.cerca.on, "Cerca", "Fuera de alcance");
  $("cercaRssi").innerHTML = (e.cerca.on ? e.cerca.rssi : 0) + "<span> dBm</span>";
  pill($("stLora"), e.lora.on, "Activo", "Sin señal", "lo");
  $("loraRssi").innerHTML = (e.lora.on ? e.lora.rssi : 0) + "<span> dBm</span>";
  $("loraSnr").textContent = e.lora.on ? "SNR " + e.lora.snr + " dB · el mínimo útil en SF9 es -12 dB" :
"SNR 0 dB · sin datos";
}
setInterval(() => {   // la base publica su estado cada 10 s; 30 s sin datos = sin enlace
  if (!DEMO && Date.now() - ultimoEstado > 30000) sinEnlace();
}, 1000);
```

## C.7 Mensajes

Muestra cada mensaje y, ante un SOS de una estación, activa la franja de alarma y la vibración.

```
// --- mensajes ---
function nombre(n) { return n == 0 ? "Web" : "Equipo " + n; }
function addMsg(m) {
  $("vacio").style.display = "none";
  const d = document.createElement("div");
  d.className = "m" + (m.de == 0 ? " web" : "") + (m.t == 1 ? " sos" : "");
  const s = document.createElement("small");
  const via = m.via == "E" ? " · ⚡ cerca" : m.via == "L" ? " · 📡 LoRa" : "";
  s.textContent = nombre(m.de) + " · " + new Date().toLocaleTimeString("es-PA", { hour: "2-digit", minute:
"2-digit" }) + via;
  d.append(s, document.createTextNode((m.t == 1 ? "🚨 " : "") + m.x));
  $("log").append(d); $("log").scrollTop = $("log").scrollHeight;
  if (m.t == 1 && m.de != 0) {
    $("alertaTxt").textContent = "🚨 " + nombre(m.de) + ": " + m.x;
    $("alerta").style.display = "flex";
    if (navigator.vibrate) navigator.vibrate([300, 150, 300]);
  }
}
$("visto").onclick = () => $("alerta").style.display = "none";
```

## C.8 Conexión MQTT

Suscripción a los temas del proyecto y publicación de mensajes (`web/msg`) y alertas (`web/sos`), que la base retransmite por radio.

```
// --- conexión MQTT ---
let client = null;
if (!DEMO) {
  client = mqtt.connect(BROKER, { clientId: "web-" + Math.random().toString(16).slice(2, 10) });
  client.on("connect", () => { pill($("conn"), true, "Conectado", ""); client.subscribe(prefix + "/#");
});
  client.on("close", () => pill($("conn"), false, "", "Desconectado"));
  client.on("message", (topic, payload, packet) => {
    const sub = topic.slice(prefix.length + 1);
    try {
      if (sub === "estado") mostrarEstado(JSON.parse(payload), !packet.retain);
      else if (sub === "radio/msg") addMsg(JSON.parse(payload));
    } catch (e) { console.warn(e); }
  });
}
// La propia base re-publica lo que enviamos (como "Web"), así que no lo dibujamos aquí
$("f").onsubmit = e => {
  e.preventDefault();
  const t = $("txt").value.trim(); if (!t) return;
  if (client) client.publish(prefix + "/web/msg", t);
  $("txt").value = "";
};
$("sos").onclick = () => {
  if (!confirm("¿Enviar SOS a las estaciones?")) return;
  if (client) client.publish(prefix + "/web/sos", "SOS desde la web");
};
```

## C.9 Modo demostración

Datos de ejemplo para visualizar el panel sin hardware.

```
// --- modo demostración (?demo) ---
if (DEMO) {
  pill($("conn"), true, "Demo", "");
  mostrarEstado({ base: true, cerca: { on: false, rssi: 0 }, lora: { on: true, rssi: -101, snr: 4 } }, tru
e);
```

```
  addMsg({ de: 2, t: 0, via: "L", x: "Llegué al cerro, ¿me copias?" });
  addMsg({ de: 1, t: 0, via: "L", x: "Fuerte y claro 👍" });
  addMsg({ de: 0, t: 0, via: "L", x: "🌐 Saludos desde la web" });
  addMsg({ de: 2, t: 2, via: "E", x: "🎤 Nota de voz" });
  addMsg({ de: 2, t: 1, via: "L", x: "SOS: botón de emergencia" });
}
</script>
</body>
</html>
```